

Análisis y Diseño de Algoritmos

Dr. Gilberto Gutiérrez R.

Universidad del Bío-Bío
Departamento de Ciencias de la Computación y TI

Primavera 2025

1 Análisis de Algoritmos

- Motivación
- Conceptos Fundamentales
- Análisis de Algoritmos no recursivos
- Análisis de Algoritmos recursivos

2 Técnicas de Diseño de Algoritmos

- Dividir para Reinar
- Programación Dinámica
- Algoritmos Voraces
- Backtracking
- Ramificación y Poda

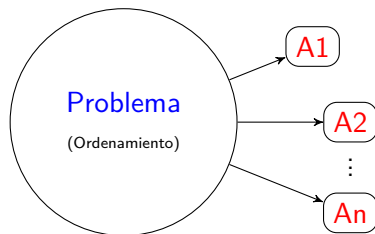
3 Complejidad Computacional

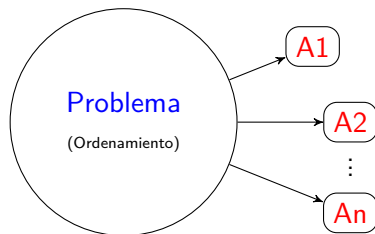
- Introducción
- Teoría de la Información
- Introducción a NP-Complejidad
- Problemas NP-Complejos

4 Algoritmos Probabilísticos

- Algoritmos Numéricos Probabilísticos
- Algoritmos de Monte Carlo
- Algoritmos de Las Vegas

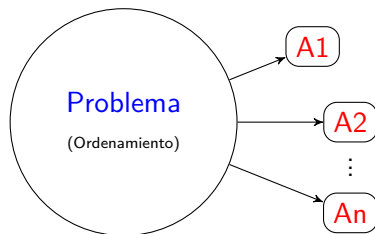
Preguntas





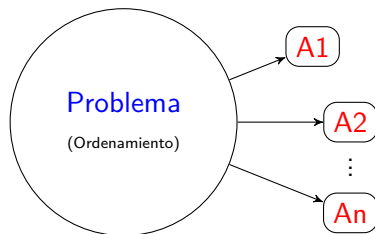
Preguntas

1. ¿ A_1 es mejor que A_2 ?



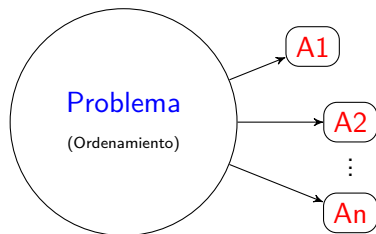
Preguntas

1. ¿ A_1 es mejor que A_2 ?
2. ¿Cuál es el mejor algoritmo?



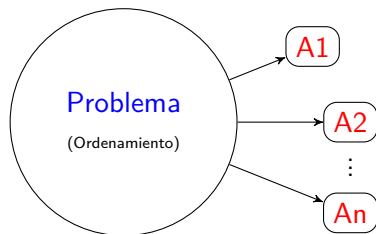
Preguntas

1. ¿ A_1 es mejor que A_2 ?
2. ¿Cuál es el mejor algoritmo?
3. ¿En qué escenarios A_2 es mejor que A_1 ?



Preguntas

1. ¿ A_1 es mejor que A_2 ?
2. ¿Cuál es el mejor algoritmo?
3. ¿En qué escenarios A_2 es mejor que A_1 ?
4. ¿Habrá un algoritmo A_{n+1} mejor que los que se conocen?



Preguntas

1. ¿ A_1 es mejor que A_2 ?
2. ¿Cuál es el mejor algoritmo?
3. ¿En qué escenarios A_2 es mejor que A_1 ?
4. ¿Habrà un algoritmo A_{n+1} mejor que los que se conocen?
5. ¿Cuál es la cota inferior del problema?
6. ¿...?

- Ej. Dado un arreglo A de tamaño n , ordenar los elementos de A de menor a mayor

Algorithm 1 SortBasico(A, n)

```
1: Input:  $A, n$ 
2: Output:  $A$  ordenado de menor a mayor
3: for  $i = 1$  to  $n - 1$  do
4:   for  $j = i + 1$  to  $n$  do
5:     if  $a[i] > a[j]$  then
6:       Intercambia( $A, i, j$ )
7:     end if
8:   end for
9: end for
```

Algorithm 2 InsertSort(A, n)

```
1: Input:  $A, n$ 
2: Output:  $A$  ordenado de menor a mayor
3: for  $j = 1$  to  $n$  do
4:    $key = a[j]$ 
5:    $i = j - 1$ 
6:   while  $i \geq 0$  and  $a[i] > key$  do
7:      $a[i + 1] = a[i]$ 
8:      $i = i - 1$ 
9:   end while
10:   $a[i + 1] = key$ 
11: end for
```

Algorithm 3 HeapSort(A, n)

```
1: Input:  $A, n$ 
2: Output:  $A$  ordenado de menor a mayor
3: BuildHeap( $A$ )
4: for  $i = n - 1$  to  $0$  do
5:   Intercambia( $A, 0, i$ )
6:   Heapify( $A, 0, i - 1$ )
7: end for
```

Algorithm 4 BuildHeap(A, n)

```
1: Input:  $A, n$   
2: Output: Un heap en  $A$   
3: for  $i = n/2; i \geq 0; i --$  do  
4:   Heapify( $A, i, n - 1$ )  
5: end for
```

Algorithm 5 Heapify(A, i, j)

```
1: Input:  $A, i, j$ 
2: Output: Heapify
3: if  $((2 * i + 1) \leq j)$  then
4:   if  $((2 * i + 2) \leq j)$  then
5:     if  $(a[2 * i + 2] \geq a[2 * i + 1])$  then
6:        $k = 2 * i + 2$ 
7:     else
8:        $k = 2 * i + 1$ 
9:     end if
10:  else
11:     $k = 2 * i + 1$ 
12:  end if
13:  if  $a[i] < a[k]$  then
14:    Intercambia( $A, i, k$ )
15:    Heapify( $A, k, j$ )
16:  end if
17: end if
```

¿Que medimos?

Recursos consumidos por el algoritmo

¿Que medimos?

Recursos consumidos por el algoritmo

- Tiempo

¿Que medimos?

Recursos consumidos por el algoritmo

- Tiempo
- Memoria (Almacenamiento)

Análisis del algoritmo (consumo de recursos)

¿Que medimos?

Recursos consumidos por el algoritmo

- Tiempo
- Memoria (Almacenamiento)

Análisis del algoritmo (consumo de recursos)

- Empírico

¿Que medimos?

Recursos consumidos por el algoritmo

- Tiempo
- Memoria (Almacenamiento)

Análisis del algoritmo (consumo de recursos)

- Empírico
- Teórico

Unidades medidas para el tiempo de ejecución

- Factores que interfieren en el tiempo de ejecución: velocidad del computador, calidad de la implementación del algoritmo (programa), compilador, etc.
- Usar una métrica que no dependa de ninguno de esos factores.
- Posible métrica, contar el número de veces que cada operación se ejecuta. Esto es muy difícil y muchas veces innecesario.
- Mejor alternativa es identificar la operación más importante del algoritmo (operación relevante / básica).

La operación relevante

- Es aquella que contribuye con la mayor parte del tiempo total de ejecución.
- La operación relevante, en la cual se basa el análisis, de alguna forma está relacionada con el tipo de problema que se intenta resolver.
- El propósito es calcular el número de veces que se ejecuta la operación relevante.
- El marco establecido para el análisis de la eficiencia de un algoritmo sugiere medirlo contando el número de veces que se ejecuta la operación relevante con entradas de tamaño n .

Midiendo el tiempo de ejecución

- Sea C_{op} el tiempo de ejecución de la operación relevante de un algoritmo sobre un computador.
- Sea $C(n)$ el número de veces que la operación relevante se ejecuta.
- El tiempo de ejecución estimado $T(n)$ de un programa que implementa el algoritmo es:

$$T(n) \approx C_{op} \cdot C(n)$$

- C_{op} y $C(n)$ son valores aproximados.
- Sin embargo a nosotros nos interesa conocer $C(n)$

Medidas de la eficiencia

Órdenes de crecimiento de las funciones

Valores de varias funciones importantes usadas en el análisis de algoritmos

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	3.3×10^1	10^2	10^3	10^3	3.6×10^6
10^2	6.6	10^2	6.6×10^2	10^4	10^6	1.3×10^{30}	9.3×10^{157}
10^3	10	10^3	1.1×10^4	10^6	10^9		
10^4	13	10^4	1.3×10^5	10^8	10^{12}		
10^5	17	10^5	1.7×10^6	10^{10}	10^{15}		
10^6	20	10^6	2.0×10^7	10^{12}	10^{18}		

Medidas de la eficiencia

Órdenes de crecimiento

- Es importante conocer el orden de crecimiento de una función para valores grandes de n .
- En el caso de las funciones 2^n y $n!$, en ambas sus valores crecen de forma astronómica a medida que el valor de n crece.
- Por ejemplo, si un computador moderno es capaz de ejecutar 10^{12} operaciones (1 billón aproximadamente) por segundo, entonces podrá demorar 4×10^{10} años en ejecutar 2^{100} operaciones ($2^{100} \approx 1,2676506 \times 10^{30}$ operaciones).
- Sin embargo, $100!$ toma aun mucho más tiempo que 4,5 mil millones de años ($100! = 9,332622 \times 10^{157}, \approx 2,959355 \times 10^{138}$ años).

Análisis de la eficiencia de los algoritmos

Peor caso, mejor caso y caso promedio

Algorithm 1 SequentialSearch(int $A[]$, $int k$)

```
1:   Input:  $A$ : arreglo de tamaño  $n$ ;  $k$ : clave a buscar.  
2:   Output: retorna posición de  $i$  en  $A$   
3:    $i = 0$ ;  
4:   while ( $i < n$ ) and ( $A[i] \neq k$ )do  
5:      $i = i + 1$   
6:   end while  
7:   if  $i < n$  then  
8:     return  $i$ ;  
9:   else  
10:    return -1  
11:  end if
```

Análisis de la eficiencia de los algoritmos

Análisis para el peor caso

- El peor caso se da cuando el algoritmo se ejecuta por el tiempo más largo de entre todas las posibles entradas de tamaño n .
- Aquí hablamos de la peor entrada posible para el algoritmo.
- En el ejemplo de la búsqueda secuencial, el peor caso se da cuando el elemento no existe en el arreglo o cuando éste es el último.
- En este caso el número de comparaciones es mayor. $C_{worst}(n) = n$.
- $C_{worst}(n) = \max_{|A|=n} T(A)$; A : entrada; T : tiempo.
- Esto nos permite encontrar una cota superior para el tiempo de ejecución el cual no excederá a $C_{worst}(n)$, para cualquier entrada de tamaño n .

Análisis de la eficiencia de los algoritmos

Análisis para el mejor caso

- El mejor caso se da cuando el algoritmo se ejecuta por el tiempo más pequeño de entre todas las posibles entradas de tamaño n .
- Aquí hablamos de la mejor entrada posible para el algoritmo.
- En el ejemplo de la búsqueda secuencial, el mejor caso se da cuando el elemento buscado es el primero en el arreglo.
- En este caso el número de comparaciones es menor. $C_{best}(n) = 1$.
- $C_{best}(n) = \min_{|A|=n} T(A)$; A : entrada; T : tiempo.
- Esto nos permite encontrar una cota inferior para el tiempo de ejecución el cual no es menor a $C_{best}(n)$, para cualquier entrada de tamaño n .

Análisis de la eficiencia de los algoritmos

Análisis para el caso promedio

- Ni el peor caso y tampoco el mejor caso entregan la información necesaria sobre el comportamiento real del algoritmo.
- El análisis del caso promedio es recomendado cuando un algoritmo tiene diferentes tiempos de ejecución para el mismo tamaño de la entrada.
- Se deben construir algunos supuestos sobre las posibles entradas de tamaño n .

Análisis de la eficiencia de los algoritmos

Análisis para el caso promedio

- En el caso de la búsqueda secuencial tenemos dos supuestos estándar:
 - La probabilidad de una búsqueda exitosa es igual a p ($0 \leq p \leq 1$).
 - La probabilidad de éxito del primer acierto en la i -ésima posición del arreglo es la misma para todo i .
- En el caso de la búsqueda exitosa la probabilidad del primer acierto es $\frac{p}{n}$, mientras que en la búsqueda no exitosa la probabilidad es $(1 - p)$.

$$C_{avg}(n) = \left[1\frac{p}{n} + 2\frac{p}{n} + \dots + i\frac{p}{n} + \dots + n\frac{p}{n} \right] + n(1 - p)$$

Análisis de la eficiencia de los algoritmos

Análisis para el caso promedio

- El número promedio de comparaciones está dado por $C_{avg}(n)$

$$C_{avg}(n) = [1\frac{p}{n} + 2\frac{p}{n} + \cdots + i\frac{p}{n} + \cdots + n\frac{p}{n}] + n(1 - p)$$

$$= \frac{p}{n} [1 + 2 + \cdots + i + \cdots + n] + n(1 - p)$$

$$= \frac{p}{n} [\sum_{i=1}^n i] + n(1 - p)$$

$$= \frac{p(n+1)}{2} + n(1 - p)$$

Análisis de la eficiencia de los algoritmos

Análisis para el caso promedio

- Los dos casos están dados por:

$$p = \begin{cases} 1 & \text{búsqueda exitosa, número de comparaciones: } \frac{(n+1)}{2} \\ 0 & \text{búsqueda no exitosa, número de comparaciones: } n \end{cases}$$

- El estudio del caso promedio es mucho más difícil que el peor y el mejor de los casos.
- Distribución de la probabilidad de las entradas posibles se asume como el valor esperado del conteo de la operación relevante:

$$\text{Caso promedio} = \sum_{|A| \leq n} Pr(A) T(A)$$

Notación Asintóticas y Clases de Eficiencia Básicas

Orden de crecimiento

- El ámbito del análisis de la eficiencia se concentra en el orden de crecimiento del conteo de la operación relevante del algoritmo.
- Para comparar y “rankear” tales órdenes de crecimiento, en ciencias de la computación se usan tres notaciones:
 - O (o grande).
 - Ω (omega grande).
 - Θ (theta grande).
- Informalmente, $O(g(n))$ es el conjunto de todas las funciones con un orden de crecimiento más bajo o el mismo que $g(n)$
- Por ejemplo: $n \in O(n^2)$; $100n + 5 \in O(n^2)$; $n^3 \notin O(n^2)$

Notación Asintóticas y Clases de Eficiencia Básicas

O grande

- Definición: una función $f(n)$ es $O(g(n))$, si:

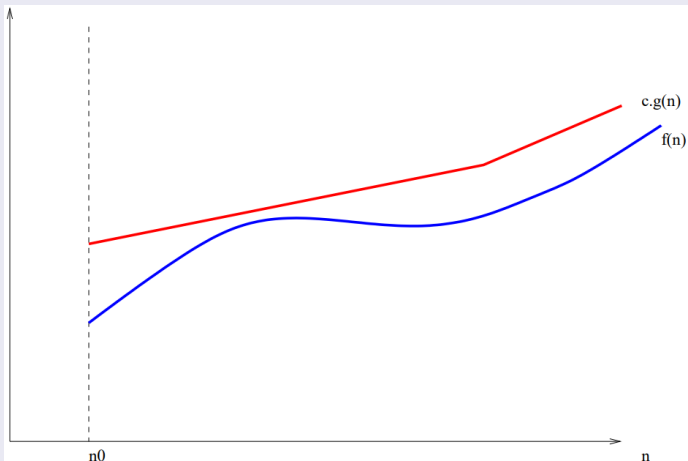
$$\exists c > 0, n_0 > 0, \text{ tal que } \forall n \geq n_0, f(n) \leq cg(n); [f(n) \preceq g(n)]$$

- En este caso podemos decir que: $f(n)$ es $O(g(n))$; o $f(n) = O(g(n))$; o $f(n) \in O(g(n))$.
- Decimos entonces que $f(n)$ está acotado por arriba (cota superior) por una constante múltiplo de $g(n)$, $\forall n$ grande.
- Por ejemplo: sea $f(n) = 100n + 5$
 $100n + 5 \leq 100n + n$ ($\forall n \geq 5$)
 $100n + 5 \leq 101n$, entonces las constantes c y n_0 son: $c = 101$, $n_0 = 5$
o, $100n + 5 \leq 100n + 5n$, ($\forall n \geq 1$), donde $c = 105$, $n_0 = 1$.

Notación Asintóticas y Clases de Eficiencia Básicas

O grande

Curvas de las funciones $f(n) \leq cg(n)$



Notación Asintóticas y Clases de Eficiencia Básicas

Ω grande

- Definición: una función $f(n)$ es $\Omega(g(n))$, si:

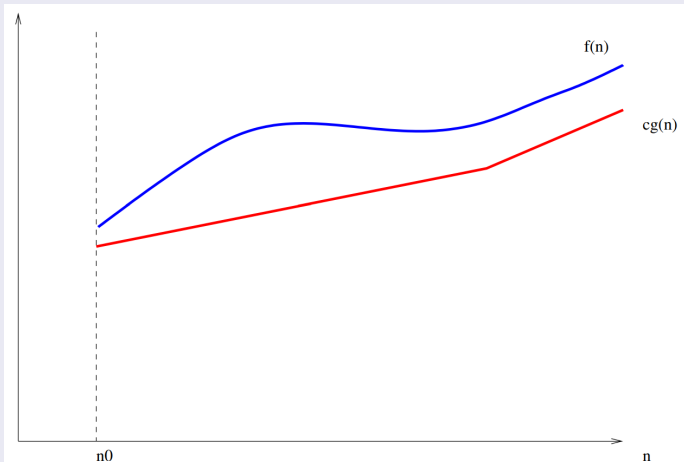
$$\exists c > 0, n_0 > 0, \text{ tal que } \forall n \geq n_0, f(n) \geq cg(n); [f(n) \succeq g(n)]$$

- En este caso podemos decir que: $f(n)$ es $\Omega(g(n))$; o $f(n) = \Omega(g(n))$; o $f(n) \in \Omega(g(n))$.
- Decimos entonces que $f(n)$ está acotado por abajo (cota inferior) por una constante múltiplo de $g(n)$, $\forall n$ grande.
- Por ejemplo: $n^3 \in \Omega(n^2)$, $n^3 \geq \Omega(n^2)$, $\forall n \geq 0$, con $c = 1$, $n_0 = 0$.
- Ω también corresponde a la complejidad del problema (ej. el problema de ordenamiento es $\Omega(n \log_2 n)$).

Notación Asintóticas y Clases de Eficiencia Básicas

Ω grande

Curvas de las funciones $f(n) \geq cg(n)$



Notación Asintóticas y Clases de Eficiencia Básicas

Θ grande

- Definición: una función $f(n)$ es $\Theta(g(n))$, si es $O(g(n))$ y $\Omega(g(n))$.
- La función $f(n)$ está acotada por arriba y por abajo por alguna constante positiva múltiplo de $g(n)$, $\forall n$ grande.
- Es decir, si existen algunas constantes positivas c_1 , c_2 y n_0 , tal que:

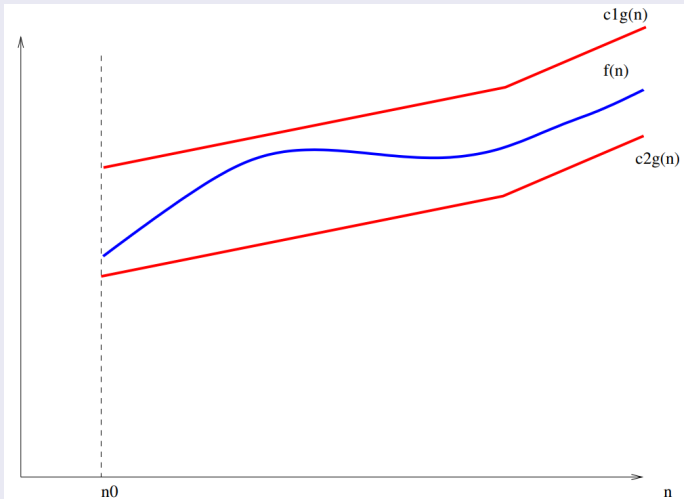
$$c_2g(n) \leq f(n) \leq c_1g(n), \forall n \geq n_0$$

- Por ejemplo, el algoritmo de sorting MergeSort es $\Theta(n \log_2 n)$.

Notación Asintóticas y Clases de Eficiencia Básicas

Θ grande

Curvas de las funciones $c_2g(n) \leq f(n) \leq c_1g(n)$



Notación Asintóticas y Clases de Eficiencia Básicas

Θ grande

- Probar que $f(n) = \frac{1}{2}n(n-1)$ es $\Theta(n^2)$.

$$f(n) = \frac{1}{2}n^2 - \frac{1}{2}n \leq \frac{1}{2}n^2$$

$$\forall n \geq 0$$

$$c_1 = \frac{1}{2}$$

$$f(n) = \frac{1}{2}n^2 - \frac{1}{2}n \geq \frac{1}{2}n^2 - \frac{1}{2}n^2$$

$$f(n) \geq \frac{1}{4}n^2$$

$$\forall n \geq 2$$

$$c_2 = \frac{1}{4}$$

$$c_1 = \frac{1}{2}, c_2 = \frac{1}{4}, n_0 = 2$$

Notación Asintóticas y Clases de Eficiencia Básicas

Regla del máximo

- Regla del máximo: Sean $f, g: \mathbb{N} \rightarrow \mathbb{R}$ dos funciones arbitrarias de los números naturales en los reales.
- La regla del máximo dice que si $T(n) = t_1(n) + t_2(n)$, siendo $t_1 = O(f(n))$ y $t_2(n) = O(g(n))$,
$$T(n) = O(f(n) + g(n)) = O(\max(f(n), g(n)))$$
- Por ejemplo: $O(n^2 + n^3 + n \log n) = O(\max(n^2, n^3, n \log n)) = O(n^3)$

Notación Asintóticas y Clases de Eficiencia Básicas

Algunas propiedades de O

Sean $f_1(n)$, $f_2(n)$, $g_1(n)$ y $g_2(n)$ funciones positivas de $N \rightarrow R$

- ① Si $f_1(n) = O(g_1(n))$ y $g_1(n) = O(g_2(n))$, entonces $f_1(n) = O(g_2(n))$
- ② Si $f_1(n) = O(g_1(n))$ y $f_2(n) = O(g_2(n))$, entonces $f_1(n)f_2(n) = O(g_1(n)g_2(n))$
- ③ $f_2(n)O(g_1(n)) = O(f_2(n)g_1(n))$
- ④ Si $f_1(n) = O(g_1(n))$ y $f_2(n) = O(g_2(n))$, entonces $f_1(n) + f_2(n) = O(g_1(n) + g_2(n))$
- ⑤ Si $c \neq 0$, c un *real* entonces $O(g_1(n)) = O(cg_1(n))$
- ⑥ Si $f_1(n) = O(g_1(n))$, entonces $cf_1(n) = O(g_1(n))$, con c un número real

Notación Asintóticas y Clases de Eficiencia Básicas

Ordenes de magnitud más comunes (en orden creciente)

Notación	Descripción
$O(1)$	Orden Constante
$O(\log_2 \log_2 n)$	Orden sublogarítmico
$O(\log_2 n)$	Orden logarítmico
$O(\sqrt{n})$	Orden sublineal
$O(n)$	Orden lineal
$O(n \log_2 n)$	Orden lineal logarítmico
$O(n^2)$	Orden cuadrático
$O(n^3)$	Orden cúbico
$O(c^n), n > 1$ y c una cte	Orden exponencial
$O(n!)$	Orden factorial
...	...

Notación Asintóticas y Clases de Eficiencia Básicas

o chica y ω (omega chica)

- Definición: una función $f(n)$ es $o(g(n))$ (o chica de $g(n)$), si:

$f(n) < cg(n)$; $[f(n) \prec g(n)]$, $f(n)$ es menor estricto de $g(n)$.

- Definición: una función $f(n)$ es $\omega(g(n))$ (omega chica de $g(n)$), si:

$f(n) > cg(n)$; $[f(n) \succ g(n)]$, $f(n)$ es mayor estricto de $g(n)$.

Notación Asintóticas y Clases de Eficiencia Básicas

Usando límite para comparar órdenes de crecimiento

- ¿Cómo comparar el orden de crecimiento entre dos funciones?
- Calculando el límite de la razón de ambas funciones. Sea $f, g: \mathbb{N} \rightarrow \mathbb{R}$, entonces podemos distinguir tres casos importantes:

- Regla de L'Hopital: $\frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{f'(n)}{g'(n)}$

(1) $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \Rightarrow f(n) = o(g(n)), \text{ y } f(n) \notin \Theta(g(n)).$

(2) $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty \Rightarrow f(n) = \omega(g(n)), \text{ y } f(n) \notin \Theta(g(n)).$

(3) $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c \in \mathbb{R} \Rightarrow f(n) = \Theta(g(n)).$

Notación Asintóticas y Clases de Eficiencia Básicas

Usando límite para comparar órdenes de crecimiento

- Ejercicios. Comparar los órdenes de crecimiento de las siguientes funciones:

(1) $(f(n) = \frac{1}{2}n(n-1)), g(n) = n^2.$

(2) $f(n) = \log_2 n, g(n) = \sqrt{n}.$

(3) $f(n) = n!, g(n) = 2^n.$

De acuerdo con la fórmula de Stirling: $n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$

Algoritmos no recursivos

Búsqueda del mayor elemento de una lista

- Ejemplo: búsqueda del mayor elemento de una lista de tamaño n .

Algorithm 7 maxElement(int $A[]$, int n)

```
1: Input:  $A$ , arreglo de tamaño  $n$ ;  $n$ , tamaño del arreglo  $A$ .
2: Output: retorna el máximo valor en  $A$ .
3: int maxval =  $A[0]$ 
4: for ( $i = 1$ ;  $i < n$ ;  $i++$ ) do
5:     if ( $A[i] > \text{maxval}$ ) then
6:         maxval =  $A[i]$ ;
7:     end if
8: end for
9: return maxval;
```

Búsqueda del mayor elemento de una lista

Análisis previo

- Tamaño de la entrada: n .
- ¿Necesita el algoritmo de espacio de almacenamiento adicional? ¡NO!
- Operación básica: la comparación.
- Se debe notar que el número de comparaciones es igual al tamaño del arreglo menos 1.
- No existe necesidad de distinguir entre el peor, mejor y caso promedio.

Búsqueda del mayor elemento de una lista

Análisis matemático

- Sea $T(n)$ el número de veces que se ejecuta la comparación.

- $T(n) = \sum_{i=1}^{n-1} 1 = n - 1 = \Theta(n).$

- Demostración:

$$\lim_{n \rightarrow \infty} \frac{n-1}{n} = \lim_{n \rightarrow \infty} \frac{n(1 - \frac{1}{n})}{n} = \lim_{n \rightarrow \infty} (1 - 0\frac{1}{n}) = 1$$

Como $1 \in \mathbb{R} \Rightarrow f(n) = n - 1 \in \Theta(n)$

Problema de la unicidad de los elementos

El algoritmo

- Chequear si todos los elementos de un arreglo dado de tamaño n son distintos.

Algorithm 8 UniqueElements(int $A[]$, int n)

```
1: Input:  $A$ , arreglo de tamaño  $n$ ;  $n$ 
2: Output: retorna true si los elementos son distintos, false en caso
   contrario.
3: for ( $i = 0$ ;  $i < n-1$ ;  $i++$ ) do
4:     for ( $j = i+1$ ;  $j < n$ ;  $j++$ ) do
5:         if ( $A[i] == A[j]$ ) then
6:             return false;
7:         end if
8:     end for
9: end for
10: return true;
```

Problema de la unicidad de los elementos

Análisis previo

- Tamaño de la entrada: n .
- ¿Necesita el algoritmo de espacio de almacenamiento adicional? ¡NO!
- Operación básica: la comparación.
- Se debe notar que el número de comparaciones no solo depende de n sino que también si existen elementos iguales en el arreglo, y si existen, en cual posición, ¿en la segunda posición encontramos el primer elemento repetido, en la i -ésima posición, en la posición n ?
- Realizaremos sólo análisis para el peor caso.

Problema de la unicidad de los elementos

Análisis previo

- Por definición, la entrada para el peor caso es un arreglo para el cual el número de comparaciones de elementos es $C_{worst}(n)$ es el más grande entre todos los arreglos de tamaño n .
- Si revisamos con atención el loop más interno, éste nos muestra que existen dos tipos de entrada para el peor caso: (a) cuando no existen elementos repetidos; (b) cuando sólo los dos últimos elementos son iguales.
- Se realiza una comparación por cada repetición del loop más interno, es decir, por cada valor de la variable j entre los límites $i + 1$ y $n - 1$; éste se repite por cada valor de i (entre 0 y $n - 2$) del loop más externo.

Problema de la unicidad de los elementos

Análisis matemático

- Revisemos las siguientes reglas de sumatorias:

$$\sum_{i=k}^n 1 = n - k + 1, \text{ con } k \leq n.$$

$$\sum_{i=0}^n i = \sum_{i=1}^n i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2} \approx \in \Theta(n^2).$$

- De acuerdo a este análisis previo, tenemos lo siguiente:

$$T(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = \sum_{i=0}^{n-2} (n-1-i)$$

Problema de la unicidad de los elementos

Análisis matemático

$$= \sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i$$

$$= (n-1)^2 - \frac{(n-2)(n-1)}{2} = \frac{n(n-1)}{2} \approx \frac{1}{2}n^2$$

$$T(n) \in O(n^2)$$

Producto de matrices

El algoritmo

- Dadas dos matrices A y B de $n \times n$, calcular la complejidad temporal del producto $C = AB$

Algorithm 9 MatrixMultiplication(int $A[][]$, int $B[][]$, int n)

```
1: Input:  $A, B$ : matrices de tamaño  $n \times n$ .
2: Output:  $C$ : matriz resultante de tamaño  $n \times n$ .
3: for ( $i = 0$ ;  $i < n$ ;  $i++$ ) do
4:     for ( $j = 0$ ;  $j < n$ ;  $j++$ ) do
5:          $C[i][j] = 0.0$ ;
6:         for  $k = 0$ ;  $k < n$ ;  $k++$  do
7:              $C[i][j] = C[i][j] + A[i][k] * B[k][j]$ ;
8:         end for
9:     end for
10: end for
11: return  $C$ ;
```

Producto de matrices

Análisis previo

- Tamaño de la entrada: $2n^2$.
- ¿Necesita el algoritmo de espacio de almacenamiento adicional? ¡NO!
- Operación básica: el producto (línea 7).
- El conteo de la operación básica depende sólo del tamaño de las matrices de entrada, entonces no es necesario analizar por separado los tres casos.
- Existe sólo una multiplicación que se ejecuta en cada iteración del loop más interno (de 0 hasta $n - 1$).

Producto de matrices

Análisis matemático

- Para el loop más interno, el número de multiplicaciones realizadas por cada par de valores de las variables i y j :

$$\sum_{k=0}^{n-1} 1 = n.$$

- El número total de multiplicaciones $T(n)$ está dado por la siguiente expresión:

$$T(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1.$$

Producto de matrices

Análisis matemático

$$T(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1 = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} n = \sum_{i=0}^{n-1} n^2 = n^3$$

$$T(n) = n^3$$

$$\therefore T(n) \in \Theta(n^3)$$

Plan general para analizar algoritmos no recursivos

- 1 Identificar el parámetro que indica el tamaño de la entrada.
- 2 Identificar la operación básica del algoritmo.
- 3 Chequear si el número de veces que se ejecuta la operación básica depende sólo del tamaño de la entrada. Si también depende de alguna propiedad adicional, entonces analizar sólo el peor caso.
- 4 Definir la expresión que refleje la suma del número de veces que la operación básica se ejecuta, a través de alguna sumatoria o una ecuación de recurrencia.
- 5 Establecer la complejidad del algoritmo (orden de crecimiento).

Ordenamiento por conteo

El algoritmo

- Por cada elemento de una lista contar el número total de elementos menores y registrar el resultado en una tabla. Los números de esta tabla indicarán las posiciones de los elementos en la lista ordenada.

Ejemplo de ordenamiento por conteo								
A[7]		58	25	74	67	101	89	15
	count[7]	0	0	0	0	0	0	0
i=0	count[7]	2	0	1	1	1	1	0
i=1	count[7]	0	1	2	2	2	2	0
i=2	count[7]	0	0	4	2	3	3	0
i=3	count[7]	0	0	0	3	4	4	0
i=4	count[7]	0	0	0	0	6	4	0
i=5	count[7]	0	0	0	0	0	5	0
	count[7]	2	1	4	3	6	5	0
F[7]	count[7]	15	25	58	67	74	89	101

Ordenamiento por conteo

El algoritmo

Algorithm 10 CountingSort(int $A[]$, int n)

```
1: Input: A: arreglo de elementos enteros; n: tamaño del arreglo.
2: Output: Arreglo A ordenado ascendentemente.
3: for ( $i = 0$ ;  $i < n$ ;  $i++$ ) do
4:      $\text{count}[i] = 0$ ;
5: end for
6: for ( $i=0$ ;  $i < n-1$ ;  $i++$ ) do
7:     for ( $j=i+1$ ;  $j < n$ ;  $j++$ ) do
8:         if ( $A[i] < A[j]$ ) then
9:              $\text{count}[j] = \text{count}[j] + 1$ ;
10:        else
11:             $\text{count}[i] = \text{count}[i] + 1$ ;
12:        end if
13:    end for
14: end for
15: for ( $i=0$ ;  $i < n$ ;  $i++$ ) do
16:      $F[\text{count}[i]] = A[i]$ ;
17: end for
18: return F;
```

Ordenamiento por conteo

Análisis previo

- Tamaño de la entrada: n (tamaño del arreglo)
- ¿Necesita el algoritmo de espacio de almacenamiento adicional? Si, arreglos `count[]` y `F`.
- Operación básica: la comparación (línea 8).
- El conteo de la operación básica depende sólo del tamaño del arreglo.

Ordenamiento por conteo

Análisis matemático

- ¿Cuál es la eficiencia del algoritmo?. Como en alguno de los ejemplos anteriores vistos, este algoritmo debería ser cuadrático.

$$T(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = \sum_{i=0}^{n-2} (n-1-i)$$

$$T(n) = \frac{n(n-1)}{2}$$

$$T(n) = \Theta(n^2)$$

Ecuaciones de recurrencia

Se pueden considerar como técnicas avanzadas de conteo

Definition

- Una sucesión es una función $f : \mathbb{N} \rightarrow A$.
- Para indicar la imagen A se emplea a_n .
- Una sucesión se denota por $a_0, a_1, a_2, \dots$.
- A los elementos $a_0, a_1, a_2, \dots$ se les llama términos de la sucesión.
- a_n es el término general.

Ejemplo

0, 1, 1, 2, 3, 5, 8, 13, ...

Ecuación de recurrencia: $a_n = a_{n-1} + a_{n-2}$, si $n > 1$.

Una expresión en la que el término general de la sucesión se escribe en función de algunos términos anteriores se llama **ecuación de recurrencia**.

Una ecuación de recurrencia no determina de manera única una sucesión. Para ello es necesario conocer algunos términos de la sucesión, los que se denominan **condiciones de borde, de frontera o condiciones iniciales**. Por ejemplo, $a_n = a_{n-1} + a_{n-2}$, si $n > 1$, $a_0 = 0$ y $a_1 = 1$.

Ejemplo

$$a_n = 2a_{n-1} - 1, n \geq 1,$$

si $a_0 = 1$ tenemos una sucesión 1, 2, 4, 8, 16...

si $a_0 = 3$ tenemos una sucesión 6, 12, 24, 48...

Para obtener un término de la sucesión en forma recurrente, es necesario obtener todos los términos anteriores. **Esto no es práctico.** Por ejemplo a_{1000} en la serie de Fibonacci. Interesa obtener una expresión en la que el término general de la sucesión dependa solo de la posición que ocupa (n) y no de términos anteriores. A esta expresión se le llama **solución de la ecuación de recurrencia**.

Ejemplo

$$a_n = 3a_{n-1}, n \geq 1, a_0 = 5$$

$$a_0 = 5$$

$$a_1 = 3a_0 = 3 \times 5 = 15$$

$$a_2 = 3a_1 = 3 \times 3 \times 5 = 45$$

...

$$a_n = 3^n 5$$

En la solución general hay dependencia sólo de n y no términos anteriores de la recurrencia.

Cálculo del factorial de un número entero n

El algoritmo

- Sea $F(n) = n!$, para $n \geq 0$. Sabemos que:
 $n! = 1 \dots (n-1)n = (n-1)!n$, por definición $0!=1$.
- Podemos calcular $F(n) = F(n-1) * n$ con el siguiente algoritmo:

Algorithm 11 $F(\text{int } n)$

```
1: Input:  $n$ : entero  $\geq 0$ .  
2: Output: Factorial de  $n$ .  
3: if ( $n == 0$ ) then  
4:   return 1;  
5: else  
6:   return  $F(n-1) * n$ ;  
7: end if
```

Cálculo del factorial de un número entero n

Análisis previo

- Tamaño de la entrada: número de bits de n .
- ¿Necesita el algoritmo de espacio de almacenamiento adicional? ¡SI!, ¿cuál? memoria para la pila.
- Operación básica: la multiplicación.
- Para obtener la ecuación de recurrencia, denotaremos al número de multiplicaciones realizadas como $T(n)$.

Cálculo del factorial de un número entero n

Análisis matemático

- $F(n)$ es calculada de acuerdo a la fórmula $F(n) = F(n - 1) * n$
- El número de multiplicaciones $T(n)$ para calcular $F(n)$ debe satisfacer la siguiente igualdad:

$$T(n) = T(n-1) + 1 \quad (n > 0).$$

- $T(n-1)$ representa el costo del cálculo de $F(n-1)$ y la constante 1 representa el producto entre $F(n-1)$ y n .

Cálculo del factorial de un número entero n

Análisis matemático

- La ecuación de recurrencia necesita una condición inicial, la cual está dada por el caso base. Cuando n es igual 0, se realizan cero multiplicaciones ($\therefore T(0)=0$).
- De esta forma ahora podemos plantear la ecuación de recurrencia completa:

$$T(n) = T(n-1) + 1 \quad (n > 0).$$

$$T(0) = 0, \text{ condición inicial.}$$

- $T(n)$ nos da el número de multiplicaciones para calcular $F(n)$.
- Para obtener la complejidad temporal del algoritmo, necesitamos resolver la ecuación de recurrencia. ¿Cómo se hace esto?

Ejemplos de ecuaciones de recurrencia

- $T(n) = T(\frac{n}{2}) + n$
- $T(n) = 2T(n-1) + 1$
- $T(n) = 2T(\frac{n}{2}) + n$
- $T(n) = T(n-1) + T(n-2), n \geq 2$
- $T(n) = T(\sqrt{n}) + 1$
- $T(n) = T(\lfloor n/2 \rfloor) + 1$
- $T(n) = T(n-1) + 2$
- $T(n) = 64T(\frac{n}{4}) + n^3$
- $T(n) = 3T(\frac{n}{3}) + \log^2 n$
- $T(n) = T(\sqrt{n}) + \log \log n$
- $T(n) = T(\frac{n}{4}) + T(\frac{3n}{4}) + 2n$
- $T(n) = 3T(\frac{n}{9}) + \sqrt{n}$
- $T(n) = 7T(\frac{n}{2}) + n^2$
- $T(n) = 64T(\frac{n}{2}) + 2^n$

Métodos para resolver ecuaciones de recurrencia

- (1) Substitución forward.
- (2) Substitución backward.
- (3) Recurrencias telescópicas.
- (4) Recurrencias lineales homogéneas.
- (5) Cambio de variables.
- (6) Teorema maestro.

Métodos para resolver ecuaciones de recurrencia

Substitución forward: Factorial recursivo

Ecuación de recurrencia (factorial) $T(n) = T(n - 1) + 1$, $T(0) = 0$

- Usamos la misma ecuación para generar los primeros términos para encontrar algún patrón:
- Se inicia desde la condición inicial.

$$T(0) = 0$$

$$T(1) = T(0) + 1 = 0 + 1 = 1$$

$$T(2) = T(1) + 1 = 1 + 1 = 2$$

$$T(3) = T(2) + 1 = 2 + 1 = 3$$

...

$$T(k) = T(k-1) + 1 = k \text{ (el patrón)}$$

$$T(n) = n \Rightarrow \Theta(n)$$

Métodos para resolver ecuaciones de recurrencia

Substitución backward: Factorial recursivo

- Resolveremos esta ecuación a través del método de substitución backward.
- $T(n) = T(n-1) + 1$, substituímos $T(n-1) = T(n-2) + 1$
 $= [T(n-2) + 1] + 1 = T(n-2) + 2$, substituímos $T(n-2) = T(n-3) + 1$
 $= [T(n-3) + 1] + 2 = T(n-3) + 3$, substituímos $T(n-3) = T(n-4) + 1$
 $= [T(n-4) + 1] + 3 = T(n-4) + 4$, etc
- Hasta aquí ya hemos hallado el patrón: $T(n) = T(n-i) + i$

Métodos para resolver ecuaciones de recurrencia

Substitución backward: Factorial recursivo

- Tomamos ahora la condicion inicial, $T(0) = 0$, debemos substituir $i = n$ en la fórmula del patrón para obtener el último resultado de la substitución.

$$T(n) = T(n-1) + 1 = \dots = T(n-i) + i = T(n-n) + n$$

$$T(n) = T(0) + n = n$$

$$T(n) = \Theta(n)$$

- Se debe notar que la versión no recursiva de este problema también es $\Theta(n)$.

Métodos para resolver ecuaciones de recurrencia

Recurrencias telescópicas

- Las recurrencias telescópicas tienen la siguiente forma general:

$$T(n) = a_{n-1} + T(n-1)$$

$$T(n) = a_{n-1} + a_{n-2} + T(n-2) \quad (pqT(n-1) = a_{n-2} + T(n-2))$$

$$T(n) = a_{n-1} + a_{n-2} + a_{n-3} + T(n-3)$$

...

$$T(n) = a_{n-1} + a_{n-2} + a_{n-3} + \cdots + a_{n-k} + \cdots + a_0 + T(0)$$

$$T(n) = T(0) + \sum_{i=0}^{n-1} a_i$$

Métodos para resolver ecuaciones de recurrencia

Recurrencias telescópicas

- En general, si:

$$T(n) = f(n) + T(n-1)$$

- Tenemos:

$$T(n) = C + \sum_{i=1}^n f(i)$$

Donde la constante C está dada por la condición inicial de la ecuación de recurrencia.

Métodos para resolver ecuaciones de recurrencia

Recurrencias telescópicas

- En el caso del algoritmo recursivo para calcular el factorial de n :

$$T(n) = T(n-1) + 1, T(0) = 0$$

$$T(n) - T(n-1) = 1$$

$$T(n-1) - T(n-2) = 1$$

$$T(n-2) - T(n-3) = 1$$

...

$$T(1) - T(0) = 1$$

- Sumando:

$$\begin{aligned} & (T(n) - T(n-1)) + (T(n-1) - T(n-2)) + (T(n-2) - T(n-3)) + \dots \\ & + (T(2) - T(1)) + (T(1) - T(0)) = \sum_{i=1}^n 1 \end{aligned}$$

Métodos para resolver ecuaciones de recurrencia

Recurrencias telescópicas

- Entonces:

$$T(n) - T(0) = \sum_{i=1}^n 1$$

$$T(n) = T(0) + \sum_{i=1}^n 1$$

$$T(n) = 0 + \sum_{i=1}^n 1 = n \Rightarrow T(n) = n$$

- Resolver:

(a) $T(n) = T(n-1) + \frac{1}{n}, n > 0, T(0) = 1;$

(b) $T(n) = T(n-1) + \log n, n > 0, T(0) = 1;$

(c) $T(n) = T(n-1) + 2^n, n > 0, T(0) = 1;$

Métodos para resolver ecuaciones de recurrencia

Recurrencias telescópicas

- Sea $T(n) = T(n-1) + \frac{1}{n}$, $n > 0$, $T(0) = 1$

$$T(n) - T(n-1) = \frac{1}{n}$$

- Luego:

$$\begin{aligned} & (T(n) - T(n-1)) + (T(n-1) - T(n-2)) + (T(n-2) - T(n-3)) + \dots \\ & + (T(2) - T(1)) + (T(1) - T(0)) = \sum_{i=1}^n \frac{1}{i} \end{aligned}$$

Métodos para resolver ecuaciones de recurrencia

Recurrencias telescópicas

$$= T(n) - T(0) = \sum_{i=1}^n \frac{1}{i}$$

$$= T(n) = T(0) + \sum_{i=1}^n \frac{1}{i} = 1 + H_n$$

- Donde H_n es el n -ésimo número armónico:

$$\sum_{i=1}^n \frac{1}{i} = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \cdots + \frac{1}{n} = \ln n + O(1)$$

- Entonces:

$$T(n) = O(\ln n)$$

Métodos para resolver ecuaciones de recurrencia

Recurrencias lineales homogéneas

- Resolveremos ahora la ecuación del factorial a través de recurrencias lineales homogéneas.
- Dada la siguiente ecuación:

$$a_k X_{n+k} + a_{k-1} X_{n+(k-1)} + \cdots + a_1 X_{n+1} + a_0 X_n = 0 \quad (*)$$

- Las soluciones de esta ecuación son de la forma: $X_n = \lambda^n$
- Entonces de la ecuación $(*)$ tenemos:

$$a_k \lambda^{n+k} + a_{k-1} \lambda^{n+(k-1)} + \cdots + a_1 \lambda^{n+1} + a_0 \lambda^n = 0$$

Métodos para resolver ecuaciones de recurrencia

Recurrencias lineales homogéneas

- Sacamos factor común λ^n :

$$\lambda^n(a_k\lambda^k + a_{k-1}\lambda^{k-1} + \cdots + a_1\lambda + a_0) = 0$$

- El polinomio: $a_k\lambda^k + a_{k-1}\lambda^{k-1} + \cdots + a_1\lambda + a_0$, se denomina polinomi característico.
- La solución a esta ecuación consiste en encontrar las raíces del polinomio característico:

$$a_k\lambda^k + a_{k-1}\lambda^{k-1} + \cdots + a_1\lambda + a_0 = 0$$

Métodos para resolver ecuaciones de recurrencia

Recurrencias lineales homogéneas

- Sean $\lambda_1, \lambda_2, \dots, \lambda_k$, las raíces del polinomio característico. La solución general es de la forma:

$$X_n = C_1 \lambda_1^n + C_2 \lambda_2^n + \dots + C_k \lambda_k^n$$

- Resolvemos el sistema de ecuaciones lineales:

$$C_1 \lambda_1^0 + C_2 \lambda_2^0 + \dots + C_k \lambda_k^0 = X_0$$

$$C_1 \lambda_1^1 + C_2 \lambda_2^1 + \dots + C_k \lambda_k^1 = X_1$$

...

$$C_1 \lambda_1^{k-1} + C_2 \lambda_2^{k-1} + \dots + C_k \lambda_k^{k-1} = X_{k-1}$$

- Los λ_i y los X_i son conocidos

Métodos para resolver ecuaciones de recurrencia

Recurrencias lineales homogéneas

- Sean λ_1 y λ_2 las raíces de la ecuación característica:

caso 1 Si $\lambda_1, \lambda_2 \in \mathbb{R}$ y $\lambda_1 \neq \lambda_2$, entonces la solución general de la ecuación de la recurrencia se obtiene con la siguiente fórmula:

$$T(n) = C_1\lambda_1^n + C_2\lambda_2^n$$

caso 2 Si $\lambda_1 = \lambda_2$, la solución general de la ecuación de la recurrencia se obtiene con la siguiente fórmula:

$$T(n) = C_1\lambda_1^n + C_2n\lambda_1^n$$

caso 3 Si $\lambda_{1,2} = u \pm iv$

$$T(n) = \gamma^n [C_1 \cos(n\theta) + C_2 \sin(n\theta)], \text{ donde } \gamma = \sqrt{u^2 + v^2}, \theta = \arctan\left(\frac{v}{u}\right)$$

C_1 y C_2 son constantes arbitrarias.

Métodos para resolver ecuaciones de recurrencia

Recurrencias lineales homogéneas

- Sea $T(n) = T(n-1) + 1$, $T(0) = 0$, también son válidas las siguientes expresiones:

$$(1) \quad T(n+1) = T(n) + 1$$

$$(2) \quad T(n+2) = T(n+1) + 1$$

- Restando las ecuaciones (2) - (1), obtenemos:

$$\begin{array}{rcl} T(n+2) - T(n+1) & = & 1 \\ T(n+1) - T(n) & = & 1 \end{array}$$

$$T(n+2) - 2T(n+1) + T(n) = 0$$

Métodos para resolver ecuaciones de recurrencia

Recurrencias lineales homogéneas

- Obtenemos la ecuación homogénea:

$$\lambda^{n+2} - 2\lambda^{n+1} + \lambda^n = 0$$

- Sacando factor común λ^n :

$$\lambda^n(\lambda^2 - 2\lambda + 1) = 0$$

$$\lambda^2 - 2\lambda + 1 = 0 \text{ (resolver la ecuación característica)}$$

- Corresponde a una ecuación cuadrática, donde $\lambda = \lambda_1 = \lambda_2 = 1$.

Métodos para resolver ecuaciones de recurrencia

Recurrencias lineales homogéneas

- De aquí obtenemos el siguiente sistema de ecuaciones lineales:

$$T(n) = C_1\lambda^n + C_2n\lambda^n, \text{ como } \lambda = 1, \text{ reemplazamos en la ecuación:}$$
$$T(n) = C_1 + C_2n \quad (**)$$

- Sabemos que $T(0) = 0$, desde aquí obtenemos el siguiente sistema de ecuaciones lineales:

$$T(0) = C_1 + C_2 \cdot 0 = 0 \rightarrow C_1 = 0 \text{ (reemplazando en la ecuación (**))}$$
$$T(1) = 1 \text{ (porque } T(1) = T(0) + 1)$$
$$T(1) = C_1 + C_2 = 1, \text{ entonces } C_2 = 1.$$

Métodos para resolver ecuaciones de recurrencia

Cambio de variables

- En algunos casos la forma telescópica no se observa directamente. Utilizando cambio de variables podemos hacer notoria la forma anteriores y facilitar la resolución de la recurrencia.
- Por ejemplo:

$$T(n) = 2T\left(\frac{n}{2}\right) + n, \quad T(n) = 1, \quad n \leq 1$$

- Supongamos que $n = 2^k$, entonces tenemos que:

$$T(2^k) = 2T(2^{k-1}) + 2^k$$

Métodos para resolver ecuaciones de recurrencia

Cambio de variables

- La forma telescópica aún no es notoria, pero veremos qué pasa si hacemos:

$T(2^k) = y(k)$, tendremos:

$$y(k) = 2y(k-1) + 2^k \text{ / dividiendo por } 2^k$$

$$\frac{y(k)}{2^k} = \frac{2y(k-1)}{2^k} + \frac{2^k}{2^k} = \frac{y(k-1)}{2^{k-1}} + 1 = \frac{y(k-1)}{2^{k-1}} + 1$$

$$\frac{y(k)}{2^k} = \frac{y(k-1)}{2^{k-1}} + 1, \text{ haciendo } r(k) = \frac{y(k)}{2^k}, \text{ obtenemos:}$$

$r(k) = r(k-1) + 1$, que es la ecuación telescópica buscada.

Métodos para resolver ecuaciones de recurrencia

Cambio de variables

- Pero sabemos que:

$$r(k) = r(k-1) + 1 = r(0) + \sum_{i=1}^k 1$$

$$r(k) = 1 + k$$

Recapitulando. Sabemos que:

$$T(n) = T(2^k) = y(k) = r(k)2^k = (1 + k)2^k = 2^k + k2^k$$

Como $n = 2^k$, entonces $k = \log_2 n$

$$T(n) = n + n \log_2 n \Rightarrow T(n) = O(n \log_2 n)$$

Métodos para resolver ecuaciones de recurrencia

Cambio de variables

- Ejercicios:

$$T(n) = 2T(n-1) + 1, T(0) = 1$$

$$T(n) = 1 + T(\sqrt{n}), T(2) = 1$$

$$T(n) = \sqrt{n}T(\sqrt{n}) + cn, T(2) = 1$$

Métodos para resolver ecuaciones de recurrencia

Teorema Maestro: Caso particular

- Sea $T(n)$ una función eventualmente no decreciente que satisface la recurrencia:
- $T(n) = aT(\frac{n}{b}) + f(n)$, $T(1) = c$, $n = b^k$, $k = 1, 2, \dots$

donde $a \geq 1$, $b \geq 2$, $c > 0$. Si $f(n) \in \Theta(n^d)$, $d \geq 0$, entonces:

$$T(n) = \begin{cases} \Theta(n^d) & \text{si } a < b^d \\ \Theta(n^d \log n) & \text{si } a = b^d \\ \Theta(n^{\log_b a}) & \text{si } a > b^d \end{cases}$$

Métodos para resolver ecuaciones de recurrencia

Teorema Maestro: ejercicios

$$(1) \quad T(n) = 1 + T(\sqrt{n})$$

$$(2) \quad T(n) = T(\lfloor \frac{n}{2} \rfloor) + 1$$

$$(3) \quad T(n) = T(\frac{n}{3}) + 1$$

$$(4) \quad T(n) = 2T(\frac{n}{2}) + 1$$

$$(5) \quad T(n) = 64T(\frac{n}{4}) + n^3$$

$$(6) \quad T(n) = 32T(\frac{n}{2}) + n^4$$

$$(7) \quad T(n) = 2T(\frac{n}{2}) + cn$$

Métodos para resolver ecuaciones de recurrencia

Teorema Maestro: ejercicios

$$(1) \quad T(n) = 1 + T(\sqrt{n})$$

Usamos substitución: sea $n = 2^k$, $k = \log n$

$$T(2^k) = 1 + T(2^{\frac{k}{2}}), \text{ de aquí hacemos: } y(k) = T(2^k)$$

$$y(k) = 1 + y\left(\frac{k}{2}\right) = n^0 + y\left(\frac{k}{2}\right), \quad a = 1, \quad b = 2, \quad d = 0.$$

Ahora aplicamos Teorema Maestro: $a = b^d = b^0 = 1$

$$y(k) = 1 + O(k^0 \log k) = 1 + O(\log k) = 1 + O(\log \log n)$$

$$T(n) = 1 + O(\log \log n) \Rightarrow T(n) = O(\log \log n)$$

Métodos para resolver ecuaciones de recurrencia

Teorema Maestro: ejercicios

$$(2) \quad T(n) = T(\lfloor \frac{n}{2} \rfloor) + 1$$

No es necesario usar substitución, así que aplicamos directamente el Teorema Maestro

$$a = 1, b = 2, d = 0 \Rightarrow a = b^0$$

$$T(n) = 1 + O(n^0 \log n)$$

$$T(n) = O(\lfloor \log n \rfloor)$$

Métodos para resolver ecuaciones de recurrencia

Teorema Maestro: ejercicios

$$(3) \quad T(n) = T\left(\frac{n}{3}\right) + 1$$

$$a = 1, b = 3, d = 0 \Rightarrow a = b^0$$

$$T(n) = 1 + O(n^0 \log n)$$

$$T(n) = O(\log_3 n)$$

Métodos para resolver ecuaciones de recurrencia

Teorema Maestro: ejercicios

$$(4) \quad T(n) = 2T\left(\frac{n}{2}\right) + 1$$

$$a = 2, b = 2, d = 0 \Rightarrow a > b^0$$

$$T(n) = O(n^{\log_2 2})$$

$$T(n) = O(n)$$

Métodos para resolver ecuaciones de recurrencia

Teorema Maestro: ejercicios

$$(5) \quad T(n) = 64T\left(\frac{n}{4}\right) + n^3$$

$$a = 64, b = 4, d = 3 \Rightarrow a = b^3$$

$$T(n) = O(n^3 \log n)$$

$$(6) \quad T(n) = 32T\left(\frac{n}{2}\right) + n^4$$

$$a = 32, b = 2, d = 4 \Rightarrow a > b^4 \Rightarrow 32 > 2^4$$

$$T(n) = O(n^{\log_2 32}) = O(n \log_2 2^5)$$

$$T(n) = O(n^5)$$

Métodos para resolver ecuaciones de recurrencia

Teorema Maestro: ejercicios

(7) $T(n) = 2T\left(\frac{n}{2}\right) + cn$ (algoritmo Merge Sort)

$$a = 2, b = 2, d = 1 \Rightarrow a = b^1$$

$$T(n) = O(n \log n)$$

(8) $T(n) = T\left(\frac{n}{2}\right) + c$ (Búsqueda binaria)

$$a = 1, b = 2, d = 0 \Rightarrow a = b^0 \Rightarrow 1 = 2^0$$

$$T(n) = O(n^0 \log_2 n)$$

$$T(n) = O(\log n)$$

Métodos para resolver ecuaciones de recurrencia

Teorema Maestro: Caso General

- Sea $T(n)$ una función eventualmente no decreciente que satisface la recurrencia:
- $T(n) = aT(\frac{n}{b}) + f(n)$ con $a \geq 1$ y $b > 1$

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{si } f(n) < O(n^c), \text{ con } c < \log_b a \\ \Theta(n^c \log_b^{k+1} n) & \text{si } f(n) = \Theta(n^c \log_b^k n) \\ & \text{con } c = \log_b a \text{ y para algún } k \geq 0 \\ \Theta(f(n)) & \text{si } f(n) = \Omega(n^c), \text{ con } c > \log_b a \text{ y} \\ & \exists k < 1, n_0 > 0 : a \cdot f(\frac{n}{b}) \leq k \cdot f(n), \forall n \geq n_0 \end{cases}$$

- Una de las técnicas más importante y más utilizada para el diseño de algoritmos.
- Consiste en descomponer un problema de tamaño n en problemas más pequeños, de modo que a partir de la solución de dichos problemas sea posible construir con facilidad una solución al problema original.

```
1:  $DR(X)$ 
2: if  $X$  es suficientemente pequeño then
3:   Retornar Ad-Hoc( $X$ )
4: else
5:   Descomponer  $X$  en subproblemas más pequeños  $X_1, X_2, \dots, X_k$ 
6:   for  $i = 1; i \leq k; i++$  do
7:      $Y_i \leftarrow DR(X_i)$ 
8:   end for
9:   Combinar las soluciones  $Y_i$  para obtener una solución  $Y$  para  $X$ 
10:  return  $Y$ 
11: end if
```

Búsqueda Binaria

Algoritmo

Suponemos un arreglo A ordenado de menor a mayor.

```
1: BusquedaBinaria( $A, li, ls, x$ )
2: if  $li > ls$  then
3:   return -1 { $x$  no se encuentra en el arreglo  $A$ }
4: else
5:   Sea  $pivote = \frac{li+ls}{2}$ 
6:   if  $A[pivote] = x$  then
7:     return  $pivote$ 
8:   else if  $x < A[pivote]$  then
9:     return BusquedaBinaria( $A, li, pivote - 1, x$ )
10:  else
11:    return BusquedaBinaria( $A, pivote + 1, ls, x$ )
12:  end if
13: end if
```


Mínimo y Máximo de un conjunto

Algoritmo

```
1: MinMax( $S$ )
2: if  $||S|| = 1$  then
3:   return  $(a,a)\{S = \{a\}\}$ 
4: else if  $||S|| = 2$  then
5:   return  $(\min(a, b), \max(a, b)) \{S = \{a, b\}\}$ 
6: else
7:   Dividir  $S$  en dos subconjuntos  $S_1, S_2$ 
8:    $(n_1, m_1) = \text{MinMax}(S_1)$ 
9:    $(n_2, m_2) = \text{MinMax}(S_2)$ 
10:  return  $(\min(n_1, n_2), \max(m_1, m_2))$ 
11: end if
```

Algoritmo

```
1: MergeSort( $T[1 \dots n]$ )
2: if  $n$  es suficientemente pequeño then
3:   Ad – Hoc( $T$ )
4: else
5:   Sea  $U[1 \dots 1 + \lfloor \frac{n}{2} \rfloor]$ ,  $V[1 \dots 1 + \lfloor \frac{n}{2} \rfloor]$ 
6:    $U[1 \dots 1 + \lfloor \frac{n}{2} \rfloor] \leftarrow T[1 \dots 1 + \lfloor \frac{n}{2} \rfloor]$ 
7:    $V[1 \dots 1 + \lfloor \frac{n}{2} \rfloor] \leftarrow T[1 + \lfloor \frac{n}{2} \rfloor \dots n]$ 
8:   MergeSort( $U[1 \dots 1 + \lfloor \frac{n}{2} \rfloor]$ )
9:   MergeSort( $V[1 \dots 1 + \lfloor \frac{n}{2} \rfloor]$ )
10:  Merge( $U, V, T$ )
11: end if
```

Sort por mezcla (continuación)

```
1: Merge( $U[1 \dots m + 1]$ ,  $V[1 \dots n + 1]$ ,  $T[1 \dots n + m]$ ) {  $U[m + 1]$  y  
    $V[n + 1]$  se usan como centinelas para facilitar el algoritmo de mezcla }  
2:  $i \leftarrow j \leftarrow 1$   
3:  $U[m + 1] \leftarrow V[n + 1] \leftarrow \infty$   
4: for  $k \leftarrow 1$  hasta  $m + n$  do  
5:   if  $U[i] < V[j]$  then  
6:      $T[k] \leftarrow U[i]$   
7:      $i \leftarrow i + 1$   
8:   else  
9:      $T[k] \leftarrow V[j]$   
10:     $j \leftarrow j + 1$   
11:   end if  
12: end for
```

Multiplicación de enteros grandes

Sea u y v dos números enteros de n bits cada uno.

$$u = a2^{n/2} + b \text{ y } v = c2^{n/2} + d$$

$$uv = (a2^{n/2} + b)(c2^{n/2} + d) = ac2^n + (ad + bc)2^{n/2} + bd$$

ac , ad , bc y bd se multiplican recursivamente.

$$T(n) = 4T(n/2) + cn, T(1) = 1$$

El término cn incluye dos desplazamientos (las multiplicaciones por 2^n y $2^{n/2}$) y las 3 sumas de a lo más $2n$ bits. Usando Teorema maestro:

$a = 4$, $b = 2$, $d = 1$ y por lo tanto $T(n) = O(n^2)$. Es posible expresar $ad + bc$ como $(a - b)(d - c) + ac + bd$. Es posible escribir

$$uv = ac2^n + ((a - b)(d - c) + ac + bd)2^{n/2} + bd$$

y ahora

$$T(n) = 3T(n/2) + bn = O(n^{1.59})$$

Multiplicación de enteros grandes

```
1: Mult(X, Y, n) {X e Y enteros con signo  $\leq 2^n$ . n potencia de 2}
2: s {contiene el signo de XY}
3: m1, m2, m3 {contiene los 3 productos}
4: A, B, C, D {contiene las mitades izquierda derecha de X e Y}
5: s = Sign(X) * Sign(Y)
6: X = abs(X)
7: Y = abs(Y)
8: if n = 1 then
9:   if X = 1 and Y = 1 then
10:    return s
11:   else
12:    return 0
13:   end if
14: else
15:   A =  $\frac{n}{2}$  bits izquierdo de X
16:   B =  $\frac{n}{2}$  bits derechos de X
17:   C =  $\frac{n}{2}$  bits izquierdo de Y
18:   D =  $\frac{n}{2}$  bits derechos de Y
19:   m1 = Mult(A, C,  $\frac{n}{2}$ )
20:   m2 = Mult(A - B, D - C,  $\frac{n}{2}$ )
21:   m3 = Mult(B, D,  $\frac{n}{2}$ )
22:   return (s * (m1 *  $2^n$  + (m1 + m2 + m3) *  $2^{\frac{n}{2}}$  + m3))
23: end if
```

Procedimiento QuickSort

```
1: quicksort( $T[i \dots j]$ ) {Ordena elementos del arreglo  $T[i \dots j]$  de menor  
   a mayor}  
2: if  $j - i$  es suficientemente pequeño then  
3:   InsertSort( $T([i \dots j])$ )  
4: else  
5:    $l = \text{pivote}(T[i \dots j])$   
6:   quicksort( $T[i \dots l - 1]$ )  
7:   quicksort( $T[l + 1 \dots j]$ )  
8: end if
```

quickSort/Partición del arreglo (pivote)

Ejemplo

3	1	4	1	5	9	2	6	5	3	5	8	9
---	---	---	---	---	---	---	---	---	---	---	---	---

El arreglo se particiona tomando $p = 3$

3	1	4	1	5	9	2	6	5	3	5	8	9
---	---	---	---	---	---	---	---	---	---	---	---	---

Se busca el primer elemento mayor que el pivote y el último no mayor que el pivote

3	1	<u>4</u>	1	5	9	2	6	5	<u>3</u>	5	8	9
---	---	----------	---	---	---	---	---	---	----------	---	---	---

Se intercambian los elementos

3	1	3	1	5	9	2	6	5	4	5	8	9
---	---	---	---	---	---	---	---	---	---	---	---	---

Se vuelve a explorar

3	1	3	1	<u>5</u>	9	<u>2</u>	6	5	4	5	8	9
---	---	---	---	----------	---	----------	---	---	---	---	---	---

Se intercambian

3	1	3	1	2	9	5	6	5	4	5	8	9
---	---	---	---	---	---	---	---	---	---	---	---	---

Se explora (los índices se cruzan)

3	1	3	1	<u>2</u>	<u>9</u>	5	6	5	4	5	8	9
---	---	---	---	----------	----------	---	---	---	---	---	---	---

Se intercambia el pivote con el elemento superrayado

2	1	3	1	3	9	5	6	5	4	5	8	9
---	---	---	---	---	---	---	---	---	---	---	---	---

Procedimiento pivote (partición)

```
1: Pivote( $T[i \dots j]$ ) { Intercambia los elementos del arreglo  $T[i \dots j]$  y retorna un valor  $l$  tal  
   que,  $i \leq l$ ,  $T[k] \forall i \leq k < l$ ,  $T[l] = p$  y  $T[k] > p \forall l < k \leq j$ , y  $p$  es el valor inicial de  $T[i]$  }  
2:  $p \leftarrow T[i]$ ;  $k \leftarrow i$ ;  $l \leftarrow j + 1$   
3: repeat  
4:    $k \leftarrow k + 1$   
5: until  $T[k] > p$  or  $k \geq j$   
6: repeat  
7:    $l \leftarrow l - 1$   
8: until  $T[l] \leq p$   
9: while  $k < l$  do  
10:   Intercambiar( $T[k]$ ,  $T[l]$ )  
11:   repeat  
12:      $k \leftarrow k + 1$   
13:   until  $T[k] > p$   
14:   repeat  
15:      $l \leftarrow l - 1$   
16:   until  $T[l] \leq p$   
17: end while  
18: Intercambiar( $T[i]$ ,  $T[l]$ )  
19: return  $l$ 
```


Peor caso

Operación relevante: Comparaciones

Sea r el tamaño de uno de los subarreglos generados por el procedimiento *pivote* (partición). El costo del algoritmo es: $t(n) = t(r) + t(n - r) + n$, es decir, el costo de ordenar un arreglo de tamaño r , más el costo de ordenar un arreglo de tamaño $n - r$ y más las comparaciones necesarias para particionar el arreglo. El peor caso ocurre cuando el tamaño de r es cero, y en este caso $t(n) = t(n - 1) + n$, $t(1) = 0$. El argumento $n - 1$ se explica por que la posición del pivote es conocida (ver líneas 6 y 7 del algoritmo *quicksort*). Por lo tanto (usar telescópica)

$$t(n) = O(n^2)$$

Mejor caso

El mejor caso sucede cuando $r = n/2$ (los dos subarreglos son del mismo tamaño aproximadamente) y por lo tanto

$$T(n) = 2t(n/2) + n = O(n \log_2 n)$$

Caso promedio

Supuesto: Sea S_1 uno de los subarreglos. Asumimos que cada uno de los tamaños $(0, 1, \dots, n-1)$ son igualmente probables. Es decir, que S_1 puede ser cualquiera de esos tamaños.

Con este supuesto valor promedio de $t(|S_1|)$ es $\frac{1}{n} \sum_{j=0}^{n-1} t(j)$. Dado que tenemos que resolver dos subproblemas (ordenar dos subarreglos) en promedio del mismo tamaño.

$$t(n) = \frac{2}{n} \sum_{j=0}^{n-1} t(j) + cn \quad (1)$$

$$nt(n) = 2 \sum_{j=0}^{n-1} t(j) + cn^2 \quad (2)$$

$$(n-1)t(n-1) = 2 \sum_{j=0}^{n-2} t(j) + c(n-1)^2 \quad (3)$$

Caso promedio (continuación)

restando (2) - (3) tenemos:

$$nt(n) - (n-1)t(n-1) = 2t(n-1) + 2cn - c \quad (4)$$

$$nt(n) = (n+1)t(n-1) + 2cn \quad (5) \text{ div. por } n(n+1)$$

$$\frac{t(n)}{n+1} = \frac{t(n-1)}{n} + \frac{2c}{n+1} \quad (6)$$

$$\frac{t(n-1)}{n} = \frac{t(n-2)}{n-1} + \frac{2c}{n} \quad (7)$$

$$\frac{t(n-2)}{n-1} = \frac{t(n-3)}{n-2} + \frac{2c}{n-1} \quad (8)$$

...

$$\frac{t(2)}{3} = \frac{t(1)}{2} + \frac{2c}{3} \quad (9)$$

Caso promedio (continuación)

Sumando todas las ecuaciones de (6) a (9)

$$\frac{t(n)}{n+1} = \frac{t(1)}{2} + 2c \sum_{i=3}^{n+1} \frac{1}{i}$$

La sumatoria es $\log_e(n+1) + \gamma - \frac{3}{2}$ con $\gamma \approx 0.577$. De esta forma

$$\frac{t(n)}{n+1} = O(\log_2 n)$$

es decir,

$$t(n) = O(n \log_2 n)$$

- Al igual que la técnica Dividir para Reinar, la Programación Dinámica (PD) resuelve problemas mediante la combinación de las soluciones de subproblemas.
- “Programación” se refiere en este contexto a un método de tabular y no a escribir código computacional.
- La PD es una técnica que se aplica especialmente en aquellos problemas donde los subproblemas se solapan, es decir, los subproblemas no son independientes.
- Un algoritmo de PD resuelve cada uno de los subproblemas más pequeños solo una vez y registra su respuesta en una tabla, evitando de esta forma repetir cálculos.

Técnicas de Diseño de Algoritmos

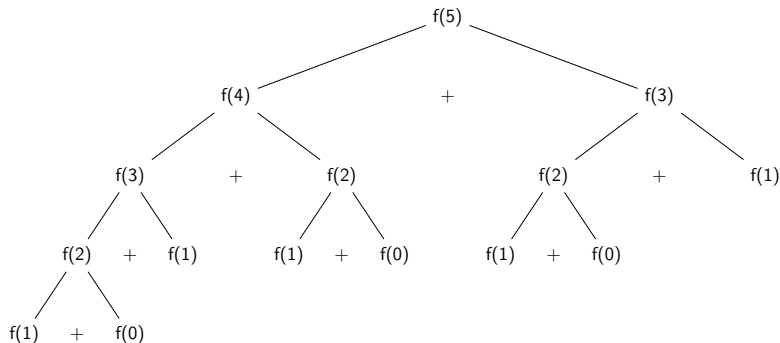
Programación Dinámica

- Donde tiene mayor aplicación la Programación Dinámica es en la resolución de problemas de optimización.
- Para que un problema pueda ser abortado por esta técnica ha de cumplir dos condiciones:
 - La solución al problema debe ser alcanzada a través de una secuencia de decisiones, una en cada etapa.
 - Dicha secuencia de decisiones ha de cumplir el principio de óptimo.

- El diseño de un algoritmo de la PD consta de los siguientes pasos:
 - (1) Planteamiento de la solución como una sucesión de decisiones y verificación de que ésta cumple el principio de óptimo.
 - (2) Definición recursiva de la solución.
 - (3) Cálculo del valor de la solución óptima mediante una tabla, donde se almacenan soluciones a problemas parciales para reutilizar los cálculos.
 - (4) Construcción de la solución óptima haciendo uso de la información contenida en la tabla.

Serie de Fibonacci

$$f(n) = \begin{cases} f(n-1) + f(n-2) & \text{si } n > 1 \\ n & \text{si } n \leq 1 \end{cases}$$



$$\text{Costo: } t(n) = \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^n - \frac{1}{\sqrt{5}} \left(\frac{1 - \sqrt{5}}{2} \right)^n$$

Técnicas de Diseño de Algoritmos

Serie Fibonacci: algoritmo no recursivo

Algorithm 12 Fibonacci(n)

```
1: Input: n: número entero.  
2: Output: el n-ésimo término de la serie de Fibonacci.  
3: vector_fibonacci[0] = 0;  
4: vector_fibonacci[1] = 1;  
5: for  $i = 2; i < n; i++$  do  
6:   vector_fibonacci[i] = vector_fibonacci[i-1] + vector_fibonacci[i-2];  
7: end for
```

- Se construye una tabla (vector) para guardar los cálculos realizados sin que éstos se repitan y después reutilizarlos.
- Es fácil ver que este algoritmo es $O(n)$.
- Su complejidad espacial también es $O(n)$.

Técnicas de Diseño de Algoritmos

Serie Fibonacci: segundo algoritmo no recursivo

- Existe otra forma mejorar este algoritmo debido a que sólo es necesario almacenar los dos últimos valores para calcular el nuevo término.
- Esto permite eliminar la tabla completa y usar sólo dos variables.
- Es fácil ver que este algoritmo también es $O(n)$.
- También podemos ver que su complejidad espacial es $O(1)$.
- Este algoritmo es el más eficiente de los tres.

Técnicas de Diseño de Algoritmos

Serie Fibonacci: segundo algoritmo no recursivo

Algorithm 13 Fibonacci(n)

```
1: Input: n: número entero.
2: Output: el n-ésimo término de la serie de Fibonacci.
3: fn_2 = 0;
4: fn_1 = 1;
5: if n == 0 then
6:     return fn_2;
7: end if
8: if n == 1 then
9:     return fn_1;
10: end if
11: for i = 2; i ≤ n; i ++ do
12:     fn = fn_1 + fn_2;
13:     aux = fn_1;
14:     fn_1 = fn;
15:     fn_2 = aux;
16: end for
17: return fn;
```

- En general, los algoritmos obtenidos mediante la aplicación de esta técnica consiguen tener complejidades (espacio y tiempo) bastante razonables.
- Se debe evitar complejidades espaciales demasiado elevadas cuando se trata de obtener complejidades temporales de orden polinómico

Técnicas de Diseño de Algoritmos

Cálculo de los Coeficientes Binomiales

- Para este problema se requiere de una tabla bidimensional.
- El algoritmo recursivo que calcula estos coeficientes es de complejidad exponencial debido a que también hay repetición de cálculos.

$$\binom{n}{m} = \begin{cases} 1 & \text{si } m = 0 \text{ o } m = n \\ \binom{n-1}{m-1} \binom{n-1}{m} & \text{si } 0 < m < n \\ 0 & \text{en caso contrario} \end{cases}$$

Técnicas de Diseño de Algoritmos

Cálculo de los Coeficientes Binomiales

Algorithm 14 $C(\text{int } n, \text{int } m)$

```
1: Input:  $n, m$ : números enteros.  
2: Output:  $n$  sobre  $m$ .  
3: if  $m = 0 \parallel m = n$  then  
4:   return 1;  
5: else  
6:   return  $C(n - 1, m - 1) + C(n - 1, m)$   
7: end if
```

- Sea $T(k)$ el tiempo necesario en el peor caso para calcular $C(n, m)$, donde $k = n + m$. La ecuación de recurrencia para el algoritmo recursivo es:

$$T(k) = T(k - 1) + T(k - 2) ; k > 1, T(1) = 0;$$

Técnicas de Diseño de Algoritmos

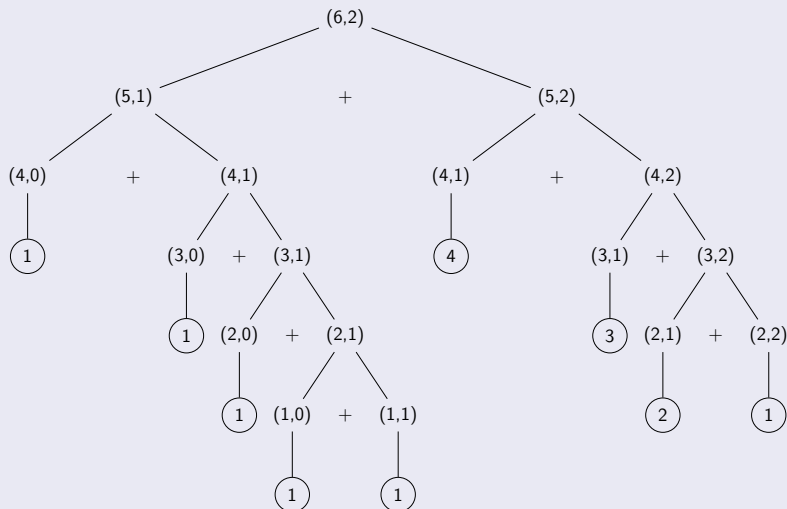
Cálculo de los Coeficientes Binomiales

$$T(k) = T(k - 1) + T(k - 2) ; k > 1, T(1) = 0;$$

- Esta ecuación de recurrencia es similar a la de la serie de Fibonacci, por lo tanto este algoritmo también tiene complejidad exponencial.

Técnicas de Diseño de Algoritmos

Cálculo de los Coeficientes Binomiales: solución no recursiva



Técnicas de Diseño de Algoritmos

Cálculo de los Coeficientes Binomiales: solución no recursiva

	0	1	2	3	...	m-1	m
0	1						
1	1	1					
2	1	2	1				
3	1	3	3	1			
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$			
n-1						$C(n-1, m-1)$	$C(n-1, m)$
n							$C(n, m)$

Diagram illustrating the recursive relationship for binomial coefficients:

$C(n-1, m-1)$ and $C(n-1, m)$ are shown above $C(n, m)$. Arrows indicate that $C(n, m)$ is calculated as the sum of $C(n-1, m-1)$ and $C(n-1, m)$.

Técnicas de Diseño de Algoritmos

Cálculo de los Coeficientes Binomiales: solución no recursiva

Algorithm 15 BC(int n, int m)

```
1: Input: n,m: números enteros.
2: Output:  $C[n+1, m+1]$ .
3: for (  $i=0$ ;  $i \leq n$ ;  $i++$  ) do
4:    $C[i,0] = 1$ ;
5: end for
6: for (  $i=1$ ;  $i \leq n$ ;  $i++$  ) do
7:    $C[i,1] = i$ ;
8: end for
9: for (  $i=2$ ;  $i \leq m$ ;  $i++$  ) do
10:   $C[i,i] = 1$ ;
11: end for
12: for (  $i=3$ ;  $i \leq n$ ;  $i++$  ) do
13:   for (  $j=2$ ;  $j \leq i-1$ ;  $j++$  ) do
14:    if (  $j \leq m$  ) then
15:       $C[i,j] = C[i-1,j-1] + C[i-1,j]$ ;
16:    end if
17:   end for
18: end for
19: return  $C[n,m]$ ;
```

Técnicas de Diseño de Algoritmos

Cálculo de los Coeficientes Binomiales: solución no recursiva

$$T(n) = \sum_{i=3}^{n-1} \sum_{j=2}^{i-2} 1 = \sum_{i=3}^{n-1} [(i-2) - 2 + 1] = \sum_{i=3}^{n-1} (i-3)$$

$$T(n) = \sum_{i=3}^{n-1} i - \sum_{i=3}^{n-1} 3 = \sum_{i=1}^{n-3} i - \sum_{i=1}^{n-3} 3$$

$$T(n) = \frac{(n-3)(n-3+1)}{2} - 3(n-3)$$

$$T(n) = \frac{n^2 - 11n + 24}{2}$$

$$T(n) = O(n^2)$$

Subsecuencia Común más Larga

Definición

Dada una secuencia $X = \{x_1, x_2, \dots, x_n\}$, decimos que $Z = \{z_1, z_2, \dots, z_k\}$, con $k \leq n$, es una subsecuencia de X si existe una secuencia creciente de índices $\{i_1, i_2, \dots, i_k\}$ de X tal que para todo $j = 1, 2, \dots, k$ se tiene que $x_{i_j} = z_j$

Ejemplo

$Z = \{ACA\}$ es una subsecuencia de $X = \{AGCAT\}$, considerando la secuencia de índices de X $\{1, 3, 4\}$

Subsecuencia Común más Larga

Dadas dos secuencias X e Y , decimos que Z es una subsecuencia común de X e Y si es subsecuencia de X y subsecuencia de Y . Deseamos determinar la longitud de la subsecuencia común más larga (L) de X e Y . Ejemplo: $X = AGCAT$ e $Y = GAC$, entonces $L(5, 3) = 2$

$$L(i, j) = \begin{cases} 0 & \text{si } i = 0 \text{ o } j = 0 \\ L(i - 1, j - 1) + 1 & \text{si } i \neq 0, j \neq 0 \text{ y } x_i = y_j \\ \max\{L(i, j - 1), L(i - 1, j)\} & \text{si } i \neq 0, j \neq 0 \text{ y } x_i \neq y_j \end{cases}$$

Técnicas de Diseño de Algoritmos

Cálculo L solución recursiva

Algorithm 16

```
1:  $L(X, i, Y, j)$ 
2: if  $i = 0$  or  $j = 0$  then
3:   return 0
4: else if  $X_i = Y_j$  then
5:   return  $L(X, i - 1, Y, j - 1) + 1$ 
6: else
7:   return  $\max\{L(X, i, Y, j - 1), L(X, i, Y, j)\}$ 
8: end if
```

Complejidad

Sea $k = n + m$ con $n = |X|$ y $m = |Y|$ y $t(k)$ el costo de resolver $L(n, m)$.

$$t(k) = 2t(k - 1) + c, T(0) = 0$$

$$t(k) = O(2^k) = O(2^{n+m})$$

Tabla

		A	G	C	A	T
	0	0	0	0	0	0
G	0	$\leftarrow \uparrow$ 0	$\nearrow$ 1	1	1	1
A	0	1	1	1	2	2
C	0	1	1	2	2	2

Algorithm 17

```
1:  $L(X, n, Y, m)$ 
2: Sea  $T[m+1][n+1]$ 
3: for  $i = 0$  to  $m$  do
4:    $T[i][0] = 0$ 
5: end for
6: for  $i = 0$  to  $n$  do
7:    $T[0][i] = 0$ 
8: end for
9: for  $i = 1$  to  $m+1$  do
10:   for  $j = 1$  to  $n+1$  do
11:     if  $X_i = Y_j$  then
12:        $T[i, j] = T[i-1, j-1] + 1$ 
13:     else
14:        $T[i, j] = \max(T[i, j-1], T[i-1, j-1])$ 
15:     end if
16:   end for
17: end for
```

Complejidad

- Almacenamiento: $O(n \cdot m)$
- Tiempo: $O(n \cdot m)$

Multiplicación encadenada de matrices

Definición

Dadas dos matrices $A_{p \times q}$ y $B_{q \times r}$ el producto de AB se define como sigue

$$C_{p \times r} = AB = \sum_{k=1}^n a_{ik} b_{kj}, 1 \leq i \leq p, 1 \leq j \leq r$$

```
1: for  $i = 1; i \leq p; i++$  do
2:   for  $j = 1; j \leq r; j++$  do
3:      $C[i,j] = 0$ 
4:     for  $k = 1; k \leq q; k++$  do
5:        $C[i,j] = C[i,j] + A[i,k] * B[k,j]$ 
6:     end for
7:   end for
8: end for
```

Se necesitan pqr multiplicaciones de escalares

Multiplicación encadenada de matrices

Supongamos ahora que se desea calcular el producto de más de dos matrices. El producto matricial es asociativo, por lo tanto es posible calcular el producto matricial

$$M = M_1 M_2 \dots M_n$$

de muchas maneras diferentes, todas las cuales darán la misma respuesta:

$$M = (\dots ((M_1 M_2) M_3) \dots M_n)$$

$$M = (M_1 (M_2 (M_3 \dots (M_{n-1} M_n) \dots)))$$

$$M = (\dots ((M_1 M_2) (M_3 M_4)) \dots)$$

y así sucesivamente. Recordar que la multiplicación de matrices no es conmutativa por lo que no es posible modificar el orden de las multiplicaciones matriciales.

Multiplicación encadenada de matrices

Ejemplo

Sean $A_{13 \times 5}$, $B_{5 \times 89}$, $C_{89 \times 3}$, $D_{3 \times 34}$ y queremos calcular $ABCD$.
Una asociación es $M = ((AB)C)D$, y cuesta (en multiplicaciones de escalares).

AB	5.785 multiplicaciones
$(AB)C$	3.471 multiplicaciones
$((AB)C)D$	1.326 multiplicaciones
Total	10.582 multiplicaciones

Hay 5 formas diferentes de calcular $ABCD$.

Forma	Multiplicaciones
$((AB)C)D$	10.582
$(AB)(CD)$	54.201
$A(BC)D$	2.856
$A((BC)D)$	4.055
$A(B(CD))$	26.418

Multiplicación encadenada de matrices

Algoritmo sencillo

- 1 Encontrar todas las formas posibles de asociación
- 2 Contar el número de multiplicaciones de escalares
- 3 Y quedarse con la de menor número de multiplicaciones

Costo

Sea $T(n)$ el número de formas diferentes de poner paréntesis en un producto de n matrices. Supongamos que decidimos hacer el primer corte entre las matrices i -ésima e $(i + 1)$ -ésima del producto en la forma siguiente:

$$(M_1 M_2 \dots M_i)(M_{i+1} M_{i+2} \dots M_n)$$

Ahora hay $T(i)$ formas de poner los paréntesis en el término del lado izquierdo y $T(n - i)$ en el término del lado derecho. Cualquier forma del primer grupo se puede combinar con cualquier forma del segundo grupo. De esta forma y para este valor concreto de i hay $T(i)T(n - i)$ formas de poner los paréntesis.

Multiplicación encadenada de matrices

Costo (continuación)

Dado que i puede tomar cualquier valor entre 1 y $n - 1$ obtenemos finalmente la recurrencia siguiente:

$$T(n) = \sum_{i=1}^{n-1} T(i)T(n-i)$$

La tabla siguiente muestra algunos valores de $T(n)^a$

n	1	2	3	4	5	10	15
$T(n)$	1	1	2	5	14	4.862	2.274.440

$T(n) = \Omega(\frac{4^n}{n^2})$ y se necesita tiempo $\Omega(n)$ para calcular la cantidad de multiplicaciones escalares. Por lo tanto se requiere tiempo $\Omega(\frac{4^n}{n})$ lo que evidentemente no es práctico.

^aLos valores de $T(n)$ se llaman números de Catalan

Multiplicación encadenada de matrices

Solución con PD

Asumiendo que la mejor forma de hacer la multiplicación se logra haciendo el primer corte entre matrices i -ésima e $(i + 1)$ -ésima.

Entonces los subproductos $M_1 M_2 \dots M_i$ y $M_{i+1} M_{i+2} \dots M_n$ también deben ser óptimos.

$m_{ij}, 1 \leq i \leq j \leq n$	almacena la solución óptima, es decir, cantidad de multiplicaciones de escalares para $M_i M_{i+1} \dots M_j$. m_{1n} es la solución del problema original.
$d[0..n]$	Vector para almacenar las dimensiones de las matrices. Las dimensiones de la matriz M_i se encuentran en $d[i - 1]$ y $d[i]$. El número de multiplicaciones de escalares al multiplicar $M_i M_{i+1}$ se obtiene directamente de $d[i - 1] \times d[i + 1]$
s	La diagonal $s = j - i$ contiene los elementos de m_{ij} . $s = 0$ diagonal principal, $s = 1$ contiene los elementos de $m_{i, i + 1}$, etc.

Solución con PD (continuación)

- ① $s = 0 : m_{i,i} = 0, i = 1, 2, \dots, n$
- ② $s = 1 : m_{i,i+1} = d_{i-1} \times d_i \times d_{i+1}, i = 1, 2, \dots, n-1$
- ③ $1 < s < n :$
$$m_{i,i+s} = \min_{i \leq k < i+s} (m_{i+k} + m_{k+1,i+s} + d_{i-1} \times d_k \times d_{i+s}),$$
$$i = 1, 2, \dots, n-s$$

Multiplicación encadenada de matrices

Ejemplo

Sean $A_{13 \times 5}$, $B_{5 \times 89}$, $C_{89 \times 3}$ y $D_{3 \times 34}$ y queremos calcular $ABCD$.

$D = [13, 5, 89, 3, 34]$

	$j = 1$	2	3	4	
$i = 1$	0	5875	1530	2856	$s = 3$
2		0	1335	1845	$s = 2$
3			0	9078	$s = 1$
4				0	$s = 0$

$s = 1$	$s = 2$	$s = 3$
$m_{34} = 89 \times 3 \times 34 = 9078$ $m_{23} = 5 \times 89 \times 3 = 1335$ $m_{12} = 13 \times 5 \times 89 = 5875$	$m_{13} = \min(m_{11} + m_{23} + 13 \times 5 \times 3, m_{12} + m_{33} + 13 \times 89 \times 3) = \min(1530, 92656) = 1530$ $m_{24} = \min(m_{22} + m_{34} + 5 \times 89 \times 34, m_{23} + m_{44} + 5 \times 5 \times 34) = \min(24208, 1845) = 1845$	$m_{14} = \min(\overbrace{m_{11} + m_{24} + 13 \times 5 \times 34}^{k=1}, m_{12} + m_{34} + 13 \times 89 \times 34, m_{13} + m_{44} + 13 \times 3 \times 34) = \min(4055, 54201, 2856) = 2856$

Multiplicación encadenada de matrices

Complejidad

Temporal:

Por cada s existen $n - s$ elementos a ser calculado en la diagonal s ; por cada una de estas se debe elegir entre s posibilidades dado por los diferentes valores de k .

$$t(n) = \sum_{s=1}^{n-1} (n-s)s = n \sum_{s=1}^{n-1} s - \sum_{s=1}^{n-1} s^2 = \frac{n^2(n-1)}{2} - \frac{n(n-1)(2n-1)}{6}$$

$$t(n) = \frac{(n^3 - n)}{6} = \Theta(n^3)$$

Almacenamiento:

matriz $m(O(n^2))$ más el arreglo $D(O(n))$. Es decir $O(n^2)$

El problema de dar cambio

El problema

Este es un problema típico cuando alguien realiza una compra y el dependiente necesita darle el cambio, pues la cantidad que ha entregado el cliente es mayor que la compra o cuando el cajero de un banco necesita pagar un cheque. Por ejemplo, supongamos que $n = 15.785$ es la cantidad a enterar (devolver o pagar) y que tenemos las siguientes denominaciones $\{20.000, 10.000, 5.000, 2.000, 1.000, 500, 100, 50, 10, 5\}$.

Suponemos que las denominaciones son monedas (no hay billetes) y que tenemos una cantidad infinita de cada una de ellas. Debemos completar la cantidad n minimizando la cantidad de monedas.

El problema de dar cambio

- 1: **ConjuntoMonedas DevolverCambio(n)**{Da el cambio de n unidades utilizando el menor número posible de monedas. C especifica un arreglo con las denominaciones}
- 2: $C = \{20000, 10000, 5000, 2000, 1000, 500, 100, 50, 10, 5, 1\}$
- 3: $S \leftarrow \emptyset$ {Conjunto que contendrá la solución}
- 4: $s \leftarrow 0$ { s es la suma de los elementos de S }
- 5: **while** $s \neq n$ **do**
- 6: $x \leftarrow$ el mayor elemento de C tal que $s + x \leq n$
- 7: **if** No existe ese elemento **then**
- 8: **return** "no existe solución"
- 9: **end if**
- 10: $S \leftarrow S \cup \{\text{una moneda de valor } x\}$
- 11: $s \leftarrow s + x$
- 12: **end while**
- 13: **return** S

Características Generales

- Problema de optimización (mínima cantidad de monedas)
- Se dispone de un conjunto de candidatos (Monedas)
- Conforme avanza el algoritmo se mantienen dos conjuntos
 - Conjunto de candidatos considerados y seleccionados
 - Conjunto de candidatos considerados y rechazados
- Se consideran 4 funciones
 - Verifica si un conjunto de candidatos es solución (ignorando si es óptima). ¿Suman las monedas seleccionadas la cantidad que hay que pagar?
 - Verifica si una solución es factible. $s + x \leq n$
 - Función de selección. Elige el candidato más prometedor de los candidatos que no han sido seleccionados o rechazados.
 - Función objetivo. Da el valor de la solución. Número de monedas. No aparece explícitamente en el algoritmo.
- La función de selección suele estar relacionada con la función objetivo.
- Nunca cambian de opinión.

Esquema general Algoritmos Voraces

```
1: voraz(Conjunto  $C$ ) {  $C$  Conjunto de candidatos}
2:  $S \leftarrow \emptyset$  {Conjunto que contendrá la solución}
3: while  $C \neq \emptyset$  and not solucion( $S$ ) do
4:    $x \leftarrow \text{seleccionar}(C)$ 
5:    $C \leftarrow C - \{x\}$ 
6:   if factible( $S \cup \{x\}$ ) then
7:      $S \leftarrow S \cup \{x\}$ 
8:   end if
9: end while
10: if solucion( $S$ ) then
11:   return  $S$ 
12: else
13:   return "No hay solución"
14: end if
```

El problema de la Mochila

Nos dan n objetos y una mochila. Para $i = 1, 2, \dots, n$, el objeto i tiene un peso positivo w_i y un valor (económico). La mochila puede llevar un peso que no sobrepase W . El objetivo es llenar la mochila de tal manera que se maximice el valor de los objetos transportados, respetando la limitación de capacidad impuesta. Suponga que es posible romper los objetos en trozos más pequeños de modo que podemos decidir llevar sólo una fracción x_i del objeto i , es decir, $0 \leq x_i \leq 1$.

Solución problema de la Mochila

La función objetivo que queremos optimizar es:

maximizar $\sum_{i=1}^n x_i v_i$ con la restricción $\sum_{i=1}^n x_i w_i \leq W$, donde $v_i > 0$, $w_i > 0$ y $0 \leq x_i \leq 1$ para $1 \leq i \leq n$.

1: **Mochila**($w[1 \dots n]$, $v[1 \dots n]$, W)

2: Sea $x[1 \dots n]$ { x es un vector para almacenar la fracción de cada objeto que va en la mochila}

3: $x[i] \leftarrow 0$ para todo $1 \leq i \leq n$

4: $\text{peso} \leftarrow 0$

5: **while** $\text{peso} < W$ **do**

6: $i \leftarrow$ el mejor objeto restante

7: **if** $((\text{peso} + w[i]) \leq W)$ **then**

8: $x[i] \leftarrow 1$

9: $\text{peso} \leftarrow \text{peso} + w[i]$

10: **else**

11: $x[i] \leftarrow (W - \text{peso})/w[i]$

12: $\text{peso} \leftarrow W$

13: **end if**

14: **end while**

15: **return** x

Funciones de selección para el problema de la mochila

- 1 El objeto más valioso (Máx v_i)
- 2 El más liviano (Mín w_i)
- 3 El objeto cuyo valor por unidad de peso sea el mayor (Máx $\frac{v_i}{w_i}$)

Ejemplo: $n = 5, W = 100$

w	10	20	30	40	50
v	20	30	66	40	60
$\frac{v}{w}$	2.0	1.5	2.2	1.0	1.2

Resultados con cada una de las funciones de selección

Seleccionar:	x_i					Valor
Máx v_i	0	0	1	0.5	1	146
Mín w_i	1	1	1	1	0	156
Máx $\frac{v_i}{w_i}$	1	1	1	0	0.8	164

- Puede aplicarse a muchos problemas (optimización)
- Problemas para los cuales no se puede construir la solución tal como ocurre con la programación dinámica o algoritmos voraces. Se deben explorar una a una todas las posibilidades factibles.
- Diferente a los algoritmos ávidos, los algoritmos Backtracking pueden cambiar una decisión.
- Exploración exhaustiva (en profundidad) de un grafo implícito (espacio de estados)
 - Éxito: Se llega a una hoja (fin, si solo buscamos una solución, se continúan buscando soluciones si se está optimizando)
 - Fracaso: El algoritmo vuelve atrás eliminando los elementos añadidos a la solución

El problema de las 8 reinas

```
1: reinas( $k$ ,  $col$ ,  $diag45$ ,  $diag135$ )
2:  $\{sol[1 \dots k]$  es  $k$ -prometedor
    $col = \{sol[i] | 1 \leq i \leq k\}$ 
    $diag45 = \{sol[i] - i + 1 | 1 \leq i \leq k\}$  y
    $diag135 = \{sol[i] + i - 1 | 1 \leq i \leq k\}$ 
3: if  $k = 8$  then
4:   Escribir  $sol$  {Un vector 8-prometedor es una solución}
5: else
6:   for  $j \leftarrow 1$  to 8 do
7:     if  $j \notin col$  y  $j - k \notin diag45$  y  $j + k \notin diag135$  then
8:        $sol[k + 1] \leftarrow j$  { $sol[1 \dots k + 1]$  es  $(k + 1)$ -prometedor}
9:        $reinas(k + 1, col \cup j, diag45 \cup \{j - k\}, diag135 \cup \{j + k\})$ 
10:    end if
11:  end for
12: end if
```

Llamada: $\Rightarrow reinas(0, \emptyset, \emptyset, \emptyset)$

El problema de la mochila

- 1: **mochilaback**(i, r) {Calcula el valor de la mejor carga que se puede construir empleando elementos de los tipos i a n y cuyo peso total no sobrepase a r . Los arreglos w y v almacenan los pesos y valores respectivamente de cada tipo de objeto.}
- 2: $b \leftarrow 0$
- 3: {se prueban por turno las clases de objetos admisibles}
- 4: **for** $k \leftarrow i$ to n **do**
- 5: **if** $w[k] \leq r$ **then**
- 6: $b \leftarrow \max(b, v[k] + \text{mochilaback}(k, r - w[k]))$
- 7: **end if**
- 8: **end for**
- 9: **return** b

Llamada: $\Rightarrow \text{mochilava}(1, W)$

El caso general

```
1: vueltaatras( $v[1 \dots k]$ )
2: {  $v$  es un vector  $k$ -prometedor }
3: if  $v$  es una solución then
4:   Escribir  $v$ 
5: else
6:   for cada vector  $(k + 1)$ -prometedor  $w$  tal que
      $w[1 \dots k] = v[1 \dots k]$  do
7:     vueltaatras( $w[1 \dots k + 1]$ )
8:   end for
9: end if
```

Introducción

- Al igual que Backtracking explora un grafo dirigido implícito.
- Puede aplicarse a problemas de optimización.

Poda

- 1 En cada nodo se calcula una cota para los posibles valores que cualquier solución puede alcanzar más adelante en el grafo.
- 2 Si la cota muestra que cualquier solución debe ser necesariamente peor que la mejor solución encontrada, entonces se evita explorar esta parte del grafo.

Ramificación

- 1 Expansión de los nodos (estrategias)
 - Anchura
 - Profundidad
 - Más prometedores (colas de prioridad)

Ramificación y Poda/Problema de Asignación

Ejemplo

Búsqueda exhaustiva: $\text{Agente} = \{a, b, c\}$, $\text{Actividad} = \{1, 2, 3\}$, c_{ij} costo de que el agente i ejecute la actividad j .

	1	2	3
a	4	7	3
b	2	6	1
c	3	9	4

Problema: Asignar las tareas a los agentes de tal manera de minimizar el costo total de ejecutar las n tareas.

Una posible asignación es $a \rightarrow 1, b \rightarrow 2, c \rightarrow 3$, $\text{costo} = 4 + 6 + 4 = 14$; pero la asignación $a \rightarrow 3, b \rightarrow 2, c \rightarrow 1$, $\text{costo} = 3 + 6 + 3 = 12$ es mejor. La mejor asignación es $a \rightarrow 2, b \rightarrow 3, c \rightarrow 1$ y su costo es $\text{costo} = 7 + 1 + 3 = 11$.

En general se necesitan considerar $n!$ asignaciones.

Ramificación y Poda/Problema de Asignación

	1	2	3	4
a	11	12	18	40
b	14	15	13	22
c	11	17	19	23
d	17	14	20	28

cota superior: $a \rightarrow 1, b \rightarrow 2, c \rightarrow 3$ y $d \rightarrow 4 = 11 + 15 + 19 + 28 = 73$.

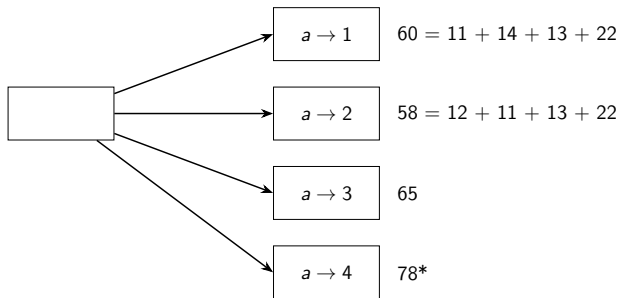
La solución no puede costar más que 73

cota inferior: Independiente de que agente ejecute la tarea 1, ésta costará 11, la tarea 2 costará 12, la tarea 3 13 y la tarea 4 22, es decir $11+12+13+22 = 58$. Es decir la solución se encuentra en el rango $[58 \dots 73]$.

Ramificación y Poda/Problema de Asignación

rango de la solución: [58...73]

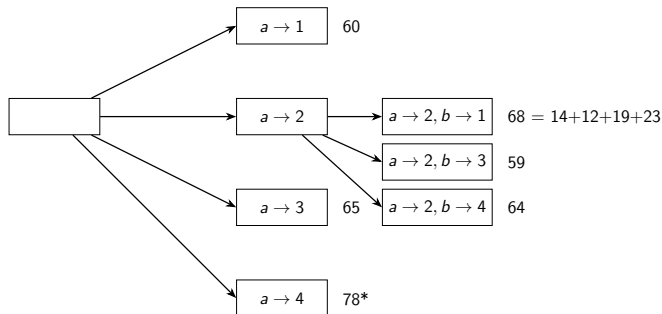
	1	2	3	4
a	11	12	18	40
b	14	15	13	22
c	11	17	19	23
d	17	14	20	28



Ramificación y Poda/Problema de Asignación

rango de la solución: [58...73]

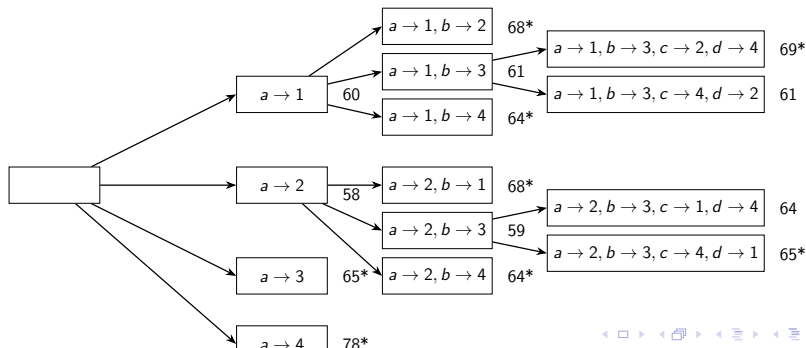
	1	2	3	4
a	11	12	18	40
b	14	15	13	22
c	11	17	19	23
d	17	14	20	28



Ramificación y Poda/Problema de Asignación

rango de la solución: [58...73]

	1	2	3	4
a	11	12	18	40
b	14	15	13	22
c	11	17	19	23
d	17	14	20	28



Algoritmo (problema de asignación)

Asignacion(A, n)

Sea $sol = cotaSuperior(A, n)$ {Cota superior del problema}

Sea ci la cota inferior de un nodo

Sea P una cola de prioridad con entradas $\langle ci, nodo \rangle$ y organizada por el mínimo de ci .

$ci = cotaInferior(A, n, r)$ {Cota inferior del nodo raíz r (cota inferior del problema)}

Insertar $\langle ci, r \rangle$ en P

while $P \neq \emptyset$ **do**

$e = EliminarMin(P)$

if $e.ci < sol$ **then**

for cada hijo h de $e.nodo$ **do**

$ci = cotaInferior(A, n, h)$

if h es un nodo hoja **then**

if $ci < sol$ **then**

$sol = ci$

end if

else

 Insertar $\langle ci, h \rangle$ en P

end if

end for

end if

end while

return sol

Complejidad Computacional

Introducción

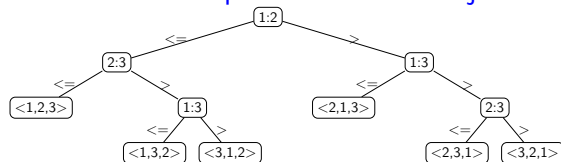
- Es una rama de la teoría o fundamentos de la computación que estudia de forma teórica el tiempo y el espacio de un problema que se considera computable y su complejidad inherente
- Considera globalmente todos los posibles algoritmos capaces de resolver un problema.
- Se busca establecer una cota inferior para el problema.
- Encontrar una función $g(n)$ tan grande como sea posible para la cual se prueba que cualquier algoritmo capaz de resolver correctamente el problema para cualquier instancia debe necesariamente tomar tiempo $\Omega(g(n))$.
- Aquí $g(n)$ representa la cota inferior de la complejidad del problema.
- Si un algoritmo es $\Theta(g(n))$ es un algoritmo eficiente para resolver el problema.

Teoría de la Información

Límite inferior para el problema de ordenamiento

Límite inferior para el problema de ordenamiento (usando comparaciones)

Árbol de decisión para ordenar tres objetos



Ejemplo: $a_1 = 6$, $a_2 = 8$, $a_3 = 5$

Sea h la altura del árbol de decisión. El peor caso ocurre cuando las $n!$ permutaciones se encuentran en el nivel h . Es decir $n! \leq 2^h$ y $h \geq \log_2 n!$.

Usando la aproximación de Stirling para $n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n (1 + \Theta(\frac{1}{n}))$, tenemos que $h = \Omega(n \log_2 n)$.

Introducción a NP-Complejidad

Introducción

- **Problemas decidibles.** Que cuentan con al menos un algoritmo que lo resuelve.
- **Problemas indecidibles.** Que no se pueden resolver mediante un algoritmo.
- Hay problemas decidibles que no pueden ser resueltos de forma eficiente, debido a que el algoritmo que los resuelve tiene tiempo exponencial o mayor.
- Esto nos permite identificar dos grupos de problemas decidibles: problemas tratables y problemas intratables.

Introducción a NP-Complejidad

problemas tratables y problemas intratables

- Problemas tratables.

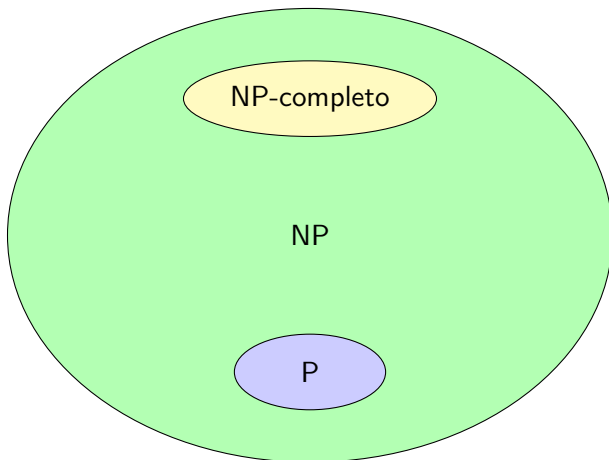
- (1) Existen algoritmos con tiempos razonables que los resuelven.
- (2) Cuentan con al menos una solución de tiempo polinomial.
- (3) Su tiempo de ejecución es conocido.

- Problemas intratables.

- (1) No existen algoritmos de tiempo razonable o eficiente que los resuelva.
- (2) Sólo se conocen algoritmos de tiempo exponencial.

clases P , NP y NP -completo

- Las clases P y NP -completo forman parte de la clase NP .
- Definiremos una clase de problemas “fáciles” y otro como “difíciles” que abstraiga del modelo de computación



- **Definición 1.** La clase P es el conjunto de todos los lenguajes que pueden decidirse en tiempo polinomial con alguna MTD (es decir, $T(n)$ es algún polinomio en n). La clase NP es el conjunto de todos los lenguajes que pueden aceptarse en tiempo polinomial con alguna $MTND$.
- En cierto sentido P representa la clase de problemas que se pueden resolver en tiempo razonable con las tecnologías conocidas y son abordables en la práctica.
- Hay muchos problemas difíciles que se resuelven fácilmente en una $MTND$ mediante “adivinar” una solución y luego verificarla en tiempo polinomial.

¿Es $P = NP$?

- Es el problema abierto más importante en computación teórica en la actualidad, y ha resistido décadas de esfuerzos.
- Hasta ahora sólo existe seguridad que $P \subseteq NP$.

Los problemas NP -completos son equivalentes

- Un paso fundamental hacia la solución del problema es la definición de la clase de problemas NP -completo.
- Los problemas NP -completos son los más difíciles dentro de la clase NP .
- Se conocen cientos de problemas NP -completos, y tras décadas de esfuerzo nadie ha logrado resolver uno de ellos en tiempo polinomial. De aquí la creencia generalizada de que $P \neq NP$.
- Todos los problemas NP -completos son equivalentes, en el sentido de que cualquiera de ellos reduce a cualquier otro.
- Para este tipo de problemas no se conocen soluciones de complejidad polinomial.

- Para explicar lo que son estos problemas debemos comenzar con el concepto de reducción polinomial.
- **Definición.** Un lenguaje $L' \subseteq \Sigma_0^*$ reduce polinomialmente a otro lenguaje $L \subseteq \Sigma_1^*$ denotado por $L' \leq L$ si existe una función $f : \Sigma_0^* \rightarrow \Sigma_1^*$ computable en tiempo polinomial en una MTD, tal que $w \in L' \iff f(w) \in L$.
- **Definición.** Un lenguaje L es *NP-completo* si:
 - (1) $L \in NP$,
 - (2) $\forall L' \in NP, L' \leq L$.Si sólo se cumple la segunda condición, entonces L es NP-hard o NP-difícil.
- **Lema 0.** Si $L' \leq L$ y $L \in P$, entonces $L' \in P$.
- **Lema 1.** Si $L'' \leq L'$ y $L' \leq L$, entonces $L'' \leq L$.

- **Lema 2.** Si L es NP-Completo y $L \in P$, entonces $P = NP$.
Prueba: Sea $L' \in NP$. Como L es NP-Completo, entonces $L' \leq L$, y si $L \in P$, por el Lema 0 $L' \in P$.
- Si se pudiera resolver cualquier problema NP-completo en tiempo polinomial, entonces inmediatamente todos los problemas NP se resolverían en tiempo polinomial.

- ¿Cómo se puede establecer que un problema es NP-completo?
- Veamos el siguiente lema:
Lema 3. Si L' es NP-completo, $L \in NP$, y $L' \leq L$, entonces L es NP-completo.
Prueba: Como L' es NP-completo, $L'' \leq L'$ para todo $L'' \in NP$. Pero $L' \leq L$, entonces por transitividad (Lema 1) $L'' \leq L$.
- Si un problema es NP-completo, en la práctica no es esperable resolverlo eficientemente, por lo que se deberá recurrir a algoritmos aproximados, probabilísticos, o meras heurísticas para tratarlo.
- Necesitamos un primer problema NP-Completo.

Primer problema NP-Completo, SAT es NP-completo

- Una fórmula proposicional es satisfactible si existe al menos una combinación de los valores de verdad de sus variables que la hace verdadera.
- Por ejemplo, si tenemos la siguiente proposición lógica:
 $P = p \vee (q \wedge r)$, veremos si esta fórmula proposicional es satisfactible:

p	q	r	p	$(q \wedge r)$	$p \vee (q \wedge r)$
0	0	0	0	0	0
0	0	1	0	0	0
0	1	0	0	0	0
0	1	1	0	1	1
1	0	0	1	0	1
1	0	1	1	0	1
1	1	0	1	0	1
1	1	1	1	1	1

- Considérese ahora la siguiente fórmula proposicional:

$$P = (p \vee \neg q \vee r) \wedge (\neg p \vee q \vee \neg r) \wedge (\neg p \vee \neg q \vee r) \wedge \\ (p \vee \neg q \vee \neg r) \wedge (\neg p \vee \neg q \vee \neg r) \wedge (p \vee q \vee r)$$

- ¿Es satisfactible?
- Sí lo es, con las asignaciones $p = 0$, $q = 0$ y $r = 1$, o $p = 1$, $q = 0$ y $r = 0$.

SAT es NP-completo

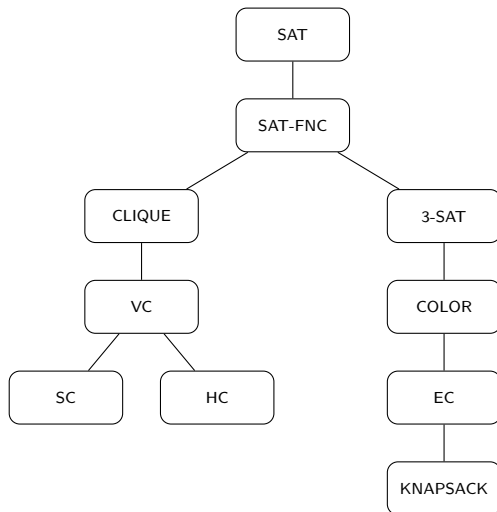
- El problema decisión de satisfactibilidad booleana (conocido como SAT) fue el primer problema identificado como perteneciente a la clase de complejidad NP-completo.
- Stephen Cook demostró en 1971 (en su publicación: “The Complexity of Theorem-Proving Procedures”, conocido como el Teorema de Cook) que SAT es NP-completo.

Metodología general para establecer que un lenguaje L es NP-completo

1. Mostrar que $L \in NP$ usando una MTND.
2. Elegir un $L' \in \text{NP-Completo}$ para mostrar que $L' \leq L$. Esto puede requerir a veces un poco de intuición y experiencia, pero a veces puede ser muy evidente también.
 - (a) Diseñar la transformación polinomial f (que es una función). Esto suele hacerse junto con la elección de L' , y es realmente la parte creativa del ejercicio.
 - (b) Mostrar que f se puede calcular en tiempo determinístico polinomial.
 - (c) Mostrar que $w \in L' \iff f(w) \in L$. Esto suele ser difícil, y normalmente va junto con el diseño de f .

Jerarquía de complejidad NP-completitud

- SAT es el primer problema NP-Completo



SAT-FNC es NP-completo

- Una fórmula proposicional está en Forma Normal Conjuntiva (FNC) si es una conjunción de disyunciones de literales. SAT-FNC es el lenguaje de las fórmulas satisfactibles que están en FNC.
- **Prueba:** Primero hay que mostrar que SAT-FNC está en NP. Una MTND puede adivinar los valores booleanos de los literales de una fórmula proposicional que está en FNC y comprobar en tiempo polinomial que la fórmula es satisfactible.
- SAT-FNC es un caso particular de SAT, así que es fácil demostrar que $SAT \leq SAT - FNC$

3-SAT es NP-completo

- **Definición.** 3-SAT es el conjunto de fórmulas proposicionales satisfactibles en FNC, donde ninguna disyunción tiene más de 3 literales.
- Por ejemplo: $P = (p \vee q \vee \neg r) \wedge (\neg p \vee q \vee r)$
- Para mostrar que 3-SAT es NP-completo, veremos que $\text{SAT-FNC} \leq 3\text{-SAT}$.

Prueba 3-SAT es NP-completo

- 3-SAT $\in$ NP ya que 3-SAT es un caso particular de SAT y también de SAT-FNC.
- La reducción comienza de la siguiente forma. Sea $F = F_1 \wedge F_2 \wedge \dots \wedge F_q$ la fórmula original en FNC.
- Transformaremos cada F_i con más de tres literales en una conjunción donde cada elemento tenga la disyunción de tres literales.
- Sea $F_i = x_1 \vee x_2 \vee \dots \vee x_k$, con x_j literal (variable o variable negada). Introduciremos variables nuevas y_1 a y_{k-3} , y reemplazaremos F_i por:

$$F'_i = (x_1 \vee x_2 \vee y_1) \wedge (\neg y_1 \vee x_3 \vee y_2) \wedge (\neg y_2 \vee x_4 \vee y_3) \wedge \dots \\ \wedge (\neg y_{k-4} \vee x_{k-2} \vee y_{k-3}) \wedge (\neg y_{k-3} \vee x_{k-1} \vee x_k)$$

- Se puede demostrar que F'_i se construye en tiempo polinomial a partir de F_i .

Prueba (cont.) 3-SAT es NP-completo

- Por ejemplo. Sea cualquier $F_i = x_1 \vee x_2 \vee x_3 \vee x_4 \vee x_5 \vee x_6 \vee x_7$
- Donde $k = 7$, y las nuevas variables y_i , con $1 \leq j \leq 4$, y_1, y_2, y_3, y_4 .
- Podemos transformar cualquier F_i a una forma normal conjuntiva compuesta de disyunciones de tres literales cada una.

$$F'_i = (x_1 \vee x_2 \vee y_1) \wedge (\neg y_1 \vee x_3 \vee y_2) \wedge (\neg y_2 \vee x_4 \vee y_3) \wedge (\neg y_3 \vee x_5 \vee y_4) \wedge (\neg y_4 \vee x_6 \vee x_7)$$

- Nuevamente podemos afirmar que F'_i se construye en tiempo polinomial a partir de F_i .

Prueba (cont.) 3-SAT es NP-completo

- $(\rightarrow)$ Si F es satisfactible, existe una asignación de valores a las variables que hace verdadero cada F_i .
- Dentro de cada F_i , al menos uno de los literales x_j se hace verdadero con esta asignación.
- Conservemos esa asignación para F'_i , y veamos que se puede asignar valores a los y_j que hagan verdadera a F'_i . Digamos $3 \leq j \leq k-2$.
- Podemos asignar $y_{j-2} = 1$ y $y_{j-1} = 0$, pues $(\neg y_{j-2} \vee x_j \vee y_{j-1})$ se mantiene verdadero.
- El valor asignado a y_{j-2} hace verdadera la disyunción anterior, $(\neg y_{j-3} \vee x_{j-1} \vee y_{j-2})$, lo cual nos permite asignar $y_{j-3} = 1$ y seguir la cadena hacia atrás.
- Similarmente, el valor asignado a y_{j-1} hace verdadera la disyunción siguiente, $(\neg y_{j-1} \vee x_{j+1} \vee y_j)$, lo que nos permite asignar y_j .

Prueba (cont.) 3-SAT es NP-completo

- ($\leftarrow$) Si $F' = F'_1 \wedge \dots \wedge F'_q$ es satisfactible, hay una asignación de variables de F' que hace verdadera cada F'_i .
- Veremos que no es posible lograr eso si ninguna de las x_j se ha hecho verdadera
- Si todas las x_j se han hecho falsas, entonces y_1 debe ser verdadera.
- Pero entonces, para hacer verdadera a $(\neg y_1 \vee x_3 \vee y_2)$ necesitamos que y_2 sea verdadera, y así siguiendo, necesitaremos finalmente que y_{k-3} sea verdadera, con lo que la última disyunción, $(\neg y_{k-3} \vee x_{k-1} \vee x_k)$ es falsa.

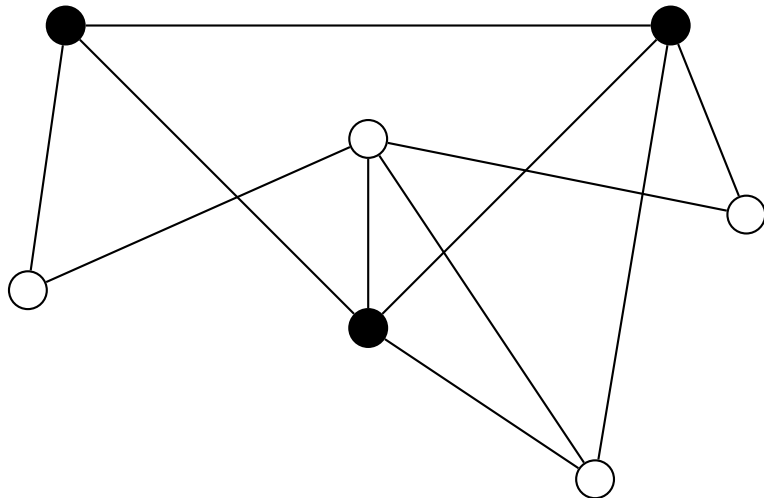
CLIQUE

Un k -clique en un grafo no dirigido $G = (V, E)$ es un subconjunto de V de tamaño k donde todos los vértices están conectados con todos. El lenguaje CLIQUE es el conjunto de pares (G, k) tal que G tiene un k -clique. Corresponde al problema de dado un grafo, determinar si tiene un k -clique.

Aplicaciones

Identificar cluster o comunidades en redes sociales (Facebook, twitter, etc.)

Ejemplo: Clique



Prueba Clique es NP-completo

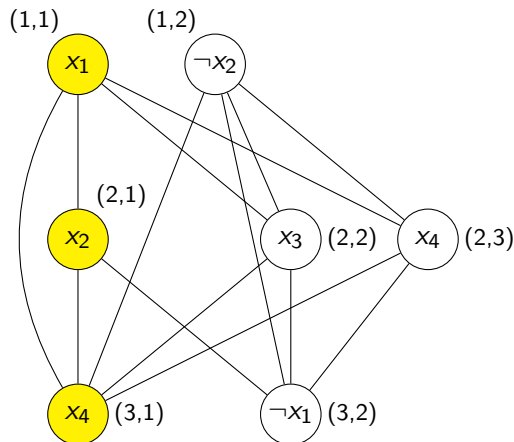
- $CLIQUE \in NP$. Una MTND puede adivinar los k vértices y luego verificar en tiempo polinomial que forman un k -Clique.
- Para demostrar que CLIQUE es NP-Completo mostraremos que $SAT-FNC \leq CLIQUE$.
 - Sea $F = F_1 \wedge F_2 \wedge \dots \wedge F_q$ una fórmula en FNC y sea $F_i = x_{i,1} \vee x_{i,2} \vee \dots \vee x_{i,m_i}$.
 - Se construye $G = (V, E)$ que tendrá un q -Clique si F es satisfactible.
 - G tendrá un vértice por cada literal de F , formalmente $V = \{(i, j), 1 \leq i \leq q, 1 \leq j \leq m_i\}$.
 - Y tendrá aristas entre literales de distintas componentes F_i y $F_{i'}$ que no sean uno la negación del otro, formalmente $E = \{((i, j), (i', j')), i \neq i', x_{i,j} \text{ no es equivalente } \neg x_{i',j'}\}$.
- Es claro que G se puede construir en tiempo polinomial.

Prueba (cont.) Clique es NP-completo

- ($\rightarrow$) Si F es satisfactible, podemos asignar valores a variables de modo que al menos un $x_{i,v(i)}$ es verdadero en cada componente de F . Estos q literales no pueden ser uno la negación del otro (todos son verdaderos a la vez). Como todos los literales están en distintas componentes y ninguno es la negación del otro, los nodos correspondientes están todos conectados en G , formando un q -Clique.
- ($\leftarrow$) Si G tiene un q -Clique, los literales asociados a los q nodos participantes deben estar todos en distintas componentes y no ser ninguno la negación del otro. Esto significa que se pueden hacer verdaderos todos a la vez, y por lo tanto tendremos un literal en cada F_i con lo que F puede hacerse verdadera.

Ejemplo 2: Clique

$$F = (x_1 \vee \neg x_2) \wedge (x_2 \vee x_3 \vee x_4) \wedge (x_4 \vee \neg x_1)$$



Un grafo con un k -clique ($k = 3$)

Problemas NP-Completos

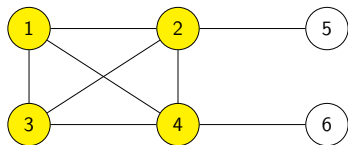
Vertex Cover

Un recubrimiento de vértices (vertex cover) de tamaño k de un grafo no dirigido $G = (V, E)$ es un subconjunto de k nodos de V tal que para toda arista $(u, v) \in E$, al menos uno entre u y v están en el subconjunto elegido. El lenguaje vc se define como los pares (G, k) tal que G tiene un recubrimiento de vértices de tamaño k .

Problemas NP-Completo

Vertex Cover es NP-Completo I

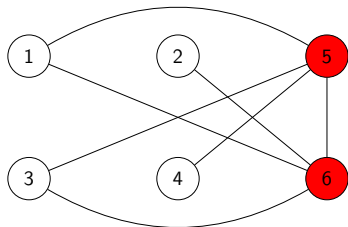
- 1 $vc \in NP$. Dado los k vértices, es fácil verificar en tiempo polinomial que por cada arista, alguno de sus vértices es alguno de los k vértices elegidos.
- 2 Para probar que vc es NP-Completo, demostramos que $CLIQUE \leq vc$.



Un grafo con un q -clique ($q = 4$)

Problemas NP-Completo

Vertex Cover es NP-Completo II



Un grafo con $k = |V| - q = 2$ vertex cover

Sea E' el complemento de las aristas de E . Entonces V' es un clique en $G = (V, E)$ ssi $V - V'$ es un recubrimiento de vértices en $G' = (V, E')$. Es decir un grafo $G = (V, E)$ dado tendrá un q -clique ssi $G' = (V, E')$ tiene un recubrimiento de vértices de tamaño $|V| - q$.

Problemas NP-Completos

Vertex Cover es NP-Completo III

- ③ $(\Rightarrow)$ Sea V' un clique en G . Sus nodos están todos conectados con todos. Si complementamos las aristas de G para formar G' , ahora esos nodos no están conectados ninguno con ninguno. Eso significa que los demás vértices, $V - V'$, cubren todas las aristas de G' , pues toda arista tiene al menos uno de sus extremos en $V - V'$ (es decir fuera de V').
- ④ $(\Leftarrow)$ Sea $V - V'$ un recubrimiento de vértices en G' . Entonces ninguna arista puede tener ambos extremos en V' , es decir, conectar dos nodos de V' . Al complementar las aristas para formar G , ahora todos los nodos de V' están conectados entre sí, formando un clique.

Problemas NP-Completo

Set Cover

Dados conjuntos $S_1, S_2, \dots, S_n$, un recubrimiento de conjuntos (set cover) de tamaño k es un grupo de k conjuntos $S_{i_1}, S_{i_2}, \dots, S_{i_k}$, tal que $\bigcup_{1 \leq j \leq k} S_{i_j} = \bigcup_{1 \leq i \leq n} S_i$. El lenguaje sc es el de los pares $(S = S_1, S_2, \dots, S_n, k)$ tal que S tiene un recubrimiento de conjuntos de tamaño k .

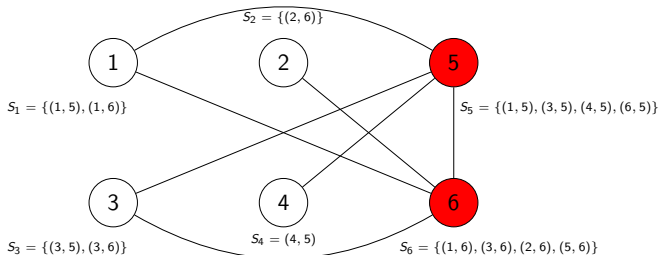
Problemas NP-Completo

Set Cover es NP-Completo I

- 1 $sc \in NP$. Dado los k conjuntos es fácil unirlos y verificar que la unión es igual a la unión de todos los conjuntos.
- 2 Para mostrar que es NP-Completo demostramos $vc \leq sc$. La reducción es muy simple. Sea un grafo $G = (V, E)$. Asociaremos a cada vértice $v \in V$ un conjunto S_v conteniendo las aristas que tocan v , formalmente $S_v = \{(u, v) \in E\}$. Evidentemente esto puede hacerse en tiempo polinomial.

Problemas NP-Completo

Set Cover es NP-Completo II



Un grafo con $k = 2$ vertex cover, con S_1, S_2, S_3, S_4, S_5 y S_6

Es fácil ver que:

- ③ ($\Rightarrow$) si $\{v_1, v_2, \dots, v_k\}$ es un recubrimiento de vértices de G entonces $S_{v_1}, S_{v_2}, \dots, S_{v_k}$ es un recubrimiento de conjuntos de S .
- ④ ($\Leftarrow$) si $S_{v_1}, S_{v_2}, \dots, S_{v_k}$ es un recubrimiento de conjuntos de S , entonces $\{v_1, v_2, \dots, v_k\}$ es un recubrimiento de vértices de G .

- Cuando un algoritmo se enfrenta a una decisión, a veces es preferible seguir un curso de acción aleatorio en lugar de invertir demasiado tiempo en determinar cuál de las alternativas es la mejor.
- Este tipo de situaciones se debe a que el tiempo para determinar la opción es prohibitivo.

Algoritmos Probabilísticos

Características de los algoritmos probabilísticos

- La característica fundamental de los algoritmos probabilísticos es que un mismo algoritmo se puede comportar de forma distinta entregando soluciones diferentes cuando se aplica dos o más veces sobre la misma entrada.
- Un algoritmo probabilístico puede no terminar o fallar pero siempre que eso ocurra con una probabilidad muy pequeña. Si ocurre, se aborta el algoritmo y su ejecución se repite con los mismos datos.
- El análisis de los algoritmos probabilísticos es en general muy difícil.
- La ejecución de un algoritmo probabilístico puede variar según el azar pero puede tomar mucho menos tiempo que la ejecución de un algoritmo determinista.

Algoritmos Probabilísticos

Características de los algoritmos probabilísticos

- Un algoritmo determinista siempre termina y entrega la misma salida ante la misma entrada.
- Un algoritmo determinista siempre encuentra la misma solución con los mismos datos de entrada.
- A un algoritmo determinista no se le permite que calcule una solución incorrecta para ningún dato. Si eso ocurre entonces existe algún error de programación.
- El análisis de la eficiencia de un algoritmo determinista es, a veces, difícil.
- La ejecución de un algoritmo determinista puede tomar demasiado tiempo.

Algoritmos Probabilísticos

Clasificación de los algoritmos probabilísticos

- **Algoritmo numéricos probabilísticos** proporcionan una solución aproximada.
- **Algoritmos de Monte Carlo** proporcionan una respuesta exacta con una alta probabilidad.
- **Algoritmos Las Vegas** nunca dan una solución incorrecta.

Algoritmos Probabilísticos

Clasificación de los algoritmos probabilísticos

Cómo se comportan los distintos tipos de algoritmos ante un mismo problema.

- ¿Cuándo descubrió América Cristobal Colón? (Brassard, Fundamentos de los Algoritmos)
- Algoritmos numéricos.
- “Entre 1490 y 1500”, “Entre 1485 y 1495”, “Entre 1491 y 1501”, “Entre 1480 y 1490”, “Entre 1489 y 1499”.
- Tenemos una probabilidad de error del 20%.
- Si se da más tiempo a la ejecución se podría reducir esa probabilidad.

Cómo se comportan los distintos tipos de algoritmos ante un mismo problema.

- ¿Cuándo descubrió América Cristobal Colón?
- Algoritmos de Monte Carlo ejecutado diez veces.
- 1492, 1492, 1492, 1491, 1492, 1492, 357 A.C., 1492, 1492, 1492.
- Nuevamente un 20% de error.
- Las respuestas incorrectas pueden ser próximas a la correcta o completamente desviadas.

Cómo se comportan los distintos tipos de algoritmos ante un mismo problema.

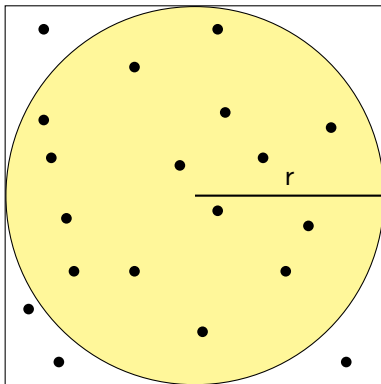
- ¿Cuándo descubrió América Cristobal Colón?
- Algoritmos Las Vegas ejecutado diez veces.
- 1492, 1492, ¡Perdón!, 1492, 1492, 1492, 1492, 1492, ¡Perdón!, 1492.
- El algoritmo puede fallar pero nunca da una respuesta incorrecta.
- El algoritmo falla con una probabilidad del 20

- Proporcionan una solución aproximada con un cierto intervalo de confianza.
- A mayor tiempo de ejecución, mejor es la aproximación

Algoritmos Probabilísticos

Algoritmos Numéricos Probabilísticos: Estimación aproximada de π (π)

- Tiramos n dardos sobre un cuadrado y contamos el número k de los que caen en un círculo inscrito en el cuadrado.



Algoritmos Probabilísticos

Algoritmos Numéricos Probabilísticos: Estimación aproximada de π (π)

- El círculo inscrito es de radio r y el lado del cuadrado es $2r$.
- Calculamos la razón R entre los dardos que caen dentro del círculo y los que caen dentro del cuadrado, que es equivalente a la razón entre el área del círculo y el área del cuadrado.

- $$R = \frac{k}{n} = \frac{\pi r^2}{(2r)^2} = \frac{\pi}{4}$$

- $$R = \frac{\pi}{4} \Rightarrow \pi = 4R$$

- $$\pi \approx 4 \frac{k}{n}$$

Algoritmos Probabilísticos

Algoritmos Numéricos Probabilísticos: Estimación aproximada de Pi (π)

- De la figura tenemos que $k=15$ y $n=19$.
- $\pi \approx 4 \frac{k}{n} = 4 \frac{15}{19} = 3,157894737$
- Es obvio que el valor que entrega es aproximado.

Algoritmos Probabilísticos

Algoritmos Numéricos Probabilísticos: Estimación aproximada de π (π)

Algorithm 18 $\text{Pi}(\text{int } n)$

```
1: Input:  $n$ , número total de dardos a lanzar.  
2: Output: valor aproximado de  $\pi$ .  
3:  $k = 0$ ;  
4: for ( $i = 0$ ;  $i < n$ ;  $i++$ ) do  
5:    $x = \text{uniforme}(0,1)$ ;  
6:    $y = \text{uniforme}(0,1)$ ;  
7:    $r = x * x + y * y$ ;  
8:   if ( $r \leq 1.0$ ) then  
9:      $k++$ ;  
10:  end if  
11: end for  
12: return  $4 * k / n$ ;
```

Algoritmos Probabilísticos

Algoritmos Numéricos Probabilísticos: Integración numérica

- Dada una función $f : [a, b] \rightarrow R$, continua, entonces el área bajo la curva $y = f(x)$, definida por: $\int_a^b f(x) dx$

Algorithm 19 IntegralDeterm(int n, double a, double b)

```
1: Input: n, cantidad de particiones
2: Output: integral aproximado de f(x) en el intervalo [a, b].
3: suma = 0;
4: delta = (b - a)/n;
5: x = a + delta/2.0;
6: for (k = 0; k < n; k++) do
7:     suma += f(x);
8:     x += delta;
9: end for
10: return suma*delta;
```

Algoritmos Probabilísticos

Algoritmos Numéricos Probabilísticos: Integración numérica

- Dada una función $f : [0, 1] \rightarrow [0, 1]$, continua, entonces el área bajo la curva $y = f(x)$, definida por: $\int_0^1 f(x) dx$

Algorithm 20 Integral(int n)

```
1: Input: n, número total de dardos a lanzar.  
2: Output: integral aproximada de f(x).  
3: k = 0;  
4: for (i = 0; i < n; i++) do  
5:     x = uniforme(0,1);  
6:     y = uniforme(0,1);  
7:     if (y ≤ f(x) ) then  
8:         k++;  
9:     end if  
10: end for  
11: return k/n;
```

Algoritmos Probabilísticos

Algoritmos Numéricos Probabilísticos: Integración numérica

- Dada una función $f : [0, 1] \rightarrow [0, 1]$, continua, entonces el área bajo la curva $y = f(x)$, definida por: $\int_0^1 f(x) dx$

Algorithm 21 Integral(int n)

```
1: Input: n, número total de dardos a lanzar.  
2: Output: integral aproximada de f(x).  
3: suma = 0;  
4: for ( $i = 0; i < n; i++$ ) do  
5:      $x = \text{uniforme}(0,1)$ ;  
6:      $\text{suma} += f(x)$ ;  
7: end for  
8: return suma/n;
```

Algorithm 22 uniforme(double a, double b)

1: **return** $a + (b-a)\text{uniforme}(0,1)$;

Algorithm 23 Integral(int n, double a, double b)

1: Input: n, número total de dardos a lanzar.
2: Output: integral aproximado de $f(x)$ en el intervalo $[a, b]$.
3: suma = 0;
4: **for** (i=0; i<n; i++) **do**
5: x = uniforme(a, b);
6: suma += f(x);
7: **end for**
8: **return** $(b-a) * \text{suma}/n$;

Algoritmos Probabilísticos

Algoritmos de Monte Carlo

- Hay problemas para los que no se conocen soluciones deterministas ni tampoco existe un algoritmo Las Vegas que nos permita resolver el problema en un tiempo razonable.
- Algoritmos que pueden entregar resultados incorrectos con una probabilidad acotada.
- Podemos disminuir esta probabilidad de error repitiendo la ejecución el algoritmo para asegurar la obtención de una respuesta correcta.
- Por lo tanto, la probabilidad de éxito es mayor cuando se dispone de más tiempo.

Algoritmos Probabilísticos

Algoritmos de Monte Carlo

- Toman decisiones probabilísticas como ayuda para llegar de forma más rápida hacia una solución correcta.
- Los algoritmos MC ocasionalmente cometen un error, pero encuentran la solución correcta con una alta probabilidad, sea cual sea el caso considerado.
- Un algoritmo Monte Carlo se dice eficiente si su tiempo de ejecución en el peor caso es polinomial para cualquier entrada.

- Sea p un número real, $0 < p < 1$, decimos que un algoritmo MC es p -correcto, si éste devuelve una respuesta correcta con probabilidad $\geq p$.

Algorithm 24 $\text{AMC}(x)$

```
1: MC(x, éxito);  
2: if (éxito == true) then  
3:   return true;  
4: else  
5:   return false;  
6: end if
```

- Búsqueda de un buen estudiante.
- Quiero un alumno con nota ≥ 5.0
- Si tomamos k estudiantes al azar, $P(error) = \frac{1}{2^k}$
- $E(T(n)) \approx \frac{1}{p(<5.0)^k}$

Algorithm 25 BuscarBuenEstudiante(x)

- 1: Elegir k alumnos al azar;
 - 2: **return** Máximo entre los k alumnos; (El resultado que retorne no siempre será correcto)
-

Algoritmos Probabilísticos

Algoritmos de Monte Carlo: Buscar un buen estudiante

- La idea de este problema es encontrar un buen estudiante, vale decir, un estudiante mejor que el del medio.
- Una solución obvia es ordenar el conjunto y sacar el máximo ($O(n \log n)$), con este, estamos seguros que es mejor que el del medio.
- Otra solución es buscar el máximo ($O(n)$).
- Qué pasa si tomamos un alumno al azar, ¿cuál es la probabilidad de que sea mejor que el del medio?. Claramente es de $1/2$.

Algoritmos Probabilísticos

Algoritmos de Monte Carlo: Buscar un buen estudiante

- Si ahora tomo dos alumnos al azar, ¿cuál es la probabilidad de que al menos uno de ellos sea mejor que el del medio?
- Qué pasa si lo vemos al revés, ¿cuál es la probabilidad de que los dos sean menores que el del medio?
- Con esta probabilidad podemos calcular la primera.
- $P(\text{al menos uno de los dos sea mejor que el del medio}) = 1 - P(\text{los dos son peores que el del medio})$.
- $P(\text{los dos son peores que el del medio}) = \frac{1}{2} * \frac{1}{2}$

Algoritmos Probabilísticos

Algoritmos de Monte Carlo: Buscar un buen estudiante

- Luego, $P(\text{al menos uno de los dos sea mejor que el del medio}) = 1 - (\frac{1}{2})^2$.
- Si tomamos un conjunto de tamaño k , tenemos que la probabilidad de que los k sean más malos que el del medio es $(1/2)^k$, luego la probabilidad de que uno de ellos sea mejor que el del medio es:

$$P(k) = 1 - \left(\frac{1}{2}\right)^k$$

- Si $k = 2$, $P(2) = 0,75$, mientras que si $k = 10$, $P(10) = 0,999023$, y con $k = 20$, $P(20) = 0,999999046$, que es suficientemente buena para casi cualquier fin práctico.

Algoritmos Probabilísticos

Algoritmos de Monte Carlo: Verificación de la multiplicación de matrices

- Consideramos tres matrices $n \times n$, A , B y C .
- Se desea verificar si la matriz C es igual al producto de A y B , es decir $C = AB$.
- ¿Cómo se puede verificar esto? La idea evidente es multiplicar A por B y comparar el resultado con C .
- Si empleamos el tradicional algoritmo de multiplicación de matrices, esto nos da $O(n^3)$.
- ¿Se puede hacer esto mejor si estamos dispuestos a admitir una pequeña probabilidad de error?

Algoritmos Probabilísticos

Algoritmos de Monte Carlo: Verificación de la multiplicación de matrices

- Sea X un vector binario de longitud n , tal que $X_i = 0$ o $X_i = 1$.
- Multiplicar XAB y XC es $\Theta(n^2)$.

Algorithm 26 testProdMat(A, B, C, n)

```
1: Input:  $A, B, C$ , matrices de  $n \times n$ .
2: Output: respuesta correcta con una probabilidad =  $\frac{1}{2}$ .
3: for ( $j=0; j<n; j++$ ) do
4:    $X_j = \text{uniforme}(0..1)$ ; /* uniforme(0..1) devuelve 0 o 1 */
5: end for
6: if  $((XA)B == XC)$  then
7:   return true;
8: else
9:   return false;
10: end if
```

Algoritmos Probabilísticos

Algoritmos de Monte Carlo: Verificación de la multiplicación de matrices

- Este algoritmo devuelve una respuesta correcta con una probabilidad de $\frac{1}{2}$.
- Esto significa que este algoritmo es $(1/2)$ -correcto.
- Por lo tanto no es p -correcto para $p < \frac{1}{2}$.
- Cuando el algoritmo retorna **false** estamos seguros que la respuesta es correcta.
- Sin embargo cuando retorna **true** no estamos seguros si la respuesta es correcta.

Algoritmos Probabilísticos

Algoritmos de Monte Carlo: Verificación de la multiplicación de matrices

- Consideremos las siguientes matrices 3×3 :

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \quad B = \begin{pmatrix} 3 & 1 & 4 \\ 1 & 5 & 9 \\ 2 & 6 & 5 \end{pmatrix} \quad C = \begin{pmatrix} 11 & 29 & 37 \\ 29 & 65 & 91 \\ 47 & 99 & 45 \end{pmatrix}$$

- Supongamos que ejecutamos la función y ésta selecciona $X = (1,1,0)$.
- Si multiplicamos $XA = (5,7,9)$, $(XA)B = (40,94,128)$, además $XC = (40,94,128)$
- Con esta selección de X la función devuelve true, ya que $XAB = XC$.
- Otra llamada a la función podría seleccionar $X=(0,1,1)$, devuelve false, $\Rightarrow XAB \neq XC$.

Algoritmos Probabilísticos

Algoritmos de Monte Carlo: Verificación de la multiplicación de matrices

- ¿Cómo podemos mejorar la probabilidad de acierto en la ejecución de la función?
- Para lograr esto, se sugiere entonces ejecutar varias veces la función (digamos k veces), donde k está en función de la probabilidad de error.

Algorithm 27 RepeatTestProdMat(A, B, C, n, k)

```
1: Input:  $A, B, C$ , matrices de  $n \times n$ ;  $k$ , número de iteraciones
2: Output: respuesta correcta con una probabilidad  $> \frac{1}{2}$ .
3: for ( $i=0$ ;  $i < k$ ;  $i++$ ) do
4:   if ( $\text{testProdMat}(A, B, C, n) == \text{false}$ ) then
5:     return false;
6:   end if
7: end for
8: return true;
```

Algoritmos Probabilísticos

Algoritmos de Monte Carlo: Verificación de la multiplicación de matrices

- Nos interesa la probabilidad de error de este nuevo algoritmo.
- Si realmente $AB=C$, entonces toda llamada a `testProdMat` devuelve necesariamente el valor verdadero y `RepeatTestProdMat` también devuelve verdadero después de k ejecuciones, por lo tanto la probabilidad de error es muy baja.
- Por otro lado, si $C \neq AB$, la probabilidad de que una llamada a `testProdMat` devuelva verdadero es como máximo $\frac{1}{2}$.
- Por lo tanto, la probabilidad de que k llamadas consecutivas devuelvan un valor erróneo es $\frac{1}{2^k}$ como máximo.

Algoritmos Probabilísticos

Algoritmos de Monte Carlo: Verificación de la multiplicación de matrices

- Podemos concluir que este nuevo algoritmo es $(1 - \frac{1}{2^k})$ -correcto.
- Cuando $k=10$, esto es mejor que 99,9%-correcto.
- Repetir 20 veces, la probabilidad de error es una entre un millón.
- Este decrecimiento rápido de la probabilidad de error es típico de los algoritmos de Monte Carlo que resuelven problemas de decisión (entregan verdadero o falso, si o no).

Algoritmos Probabilísticos

Algoritmos de Monte Carlo: Verificación de la multiplicación de matrices

- De forma alternativa, se puede dar a los algoritmos de Monte Carlo una cota superior explícita para la probabilidad de error admisible.
- Dado ε , la probabilidad máxima de error.
- Claramente el algoritmo requiere un tiempo que está en $O(n^2 \log \frac{1}{\varepsilon})$.

Algorithm 28 testProdMatEpsilon(A, B, C, n, ε)

- 1: Input: A, B, C , matrices de $n \times n$; ε , probabilidad máxima de error
 - 2: Output: respuesta correcta con una probabilidad $> \frac{1}{2}$.
 - 3: $k = \lceil \log \frac{1}{\varepsilon} \rceil$
 - 4: **return** RepeatTestProdMat(A, B, C, n, k);
-

Algoritmos Probabilísticos

Algoritmos de Monte Carlo: Verificación de la multiplicación de matrices

- Es $O(n^2 \log \frac{1}{\varepsilon})$ porque el producto de los vectores y las matrices se realiza en tiempo n^2 , y este producto se ejecuta k veces, y sabemos que $k = \lceil \log \frac{1}{\varepsilon} \rceil$.

$$E(T(n)) = O\left(n^2 \log \frac{1}{\varepsilon}\right)$$

- Esta solución podría resultar de utilidad si fuera necesario verificar productos matriciales para matrices muy grandes.

Algoritmos Probabilísticos

Algoritmos de Monte Carlo: Vector Mayoritario

- Sea $A[n]$, un arreglo de enteros, se dice que un arreglo es mayoritario si éste tiene un elemento mayoritario, es decir, un elemento x tal que su ocurrencia es $> n/2$.
- Por ejemplo:

$A[14] = \{7, 7, 1, 7, 2, 1, 7, 7, 6, 7, 7, 3, 7, 1\}$; es mayoritario.

$B[14] = \{0, 1, 1, 3, 2, 1, 7, 7, 7, 7, 7, 7, 7, 7\}$; es mayoritario.

$C[14] = \{0, 1, 1, 3, 2, 1, 4, 6, 7, 9, 4, 5, 4, 8\}$; no es mayoritario.

Algorithm 29 MayoritarioMC(int[] A, int n)

```
1: Input: A, matriz de enteros; n tamaño de A
2: Output: verdadero si A es mayoritario.
3: i = uniforme(0..n);
4: k = 0;
5: x = A[i];
6: for (j=0; j<n; j++) do
7:     if (A[j] == x) then
8:         k++;
9:     end if
10: end for
11: return k > n/2;
```

Algoritmos Probabilísticos

Algoritmos de Monte Carlo: Vector Mayoritario

- Iteramos k veces para disminuir la probabilidad de error.
- En el peor de los casos, este algoritmo es $O(n \log \frac{1}{\epsilon})$.

Algorithm 30 RepeatMayoritarioMC(int[] A, int n, float ϵ)

```
1: Input: A, matriz de enteros; n tamaño de A
2: Output: verdadero si A es mayoritario.
3:  $k = \lceil \log \frac{1}{\epsilon} \rceil$ 
4: for (i=0; i<k; i++) do
5:     if (MayoritarioMC(A, n)) then
6:         return true;
7:     end if
8: end for
9: return false;
```

Algoritmos Probabilísticos

Algoritmos de Las Vegas

- A diferencia de los algoritmos de Monte Carlo, nunca proporcionan respuestas incorrectas, es decir, nunca se equivocan.
- No garantizan terminar en un tiempo prudente.
- Se emplean para resolver problemas para los que no se conoce ningún algoritmo determinista eficiente.
- La probabilidad de éxito es mayor cuando se dispone de más tiempo para ejecutar el algoritmo.

- Toman decisiones al azar para encontrar soluciones en menos tiempo que un algoritmo determinista.
- Itera hasta que se devuelva un resultado correcto.
- El siguiente algoritmo muestra la forma general de los algoritmos de Las Vegas.

Algorithm 38 RepeatALV(x)

```
1: repeat  
2:   ALV( $x$ ,  $y$ ,  $\text{exito}$ )  
3: until  $\text{exito} == \text{true}$ ;  
4: return  $y$ ;
```

Algoritmos Probabilísticos

Algoritmos de Las Vegas

- Puesto que cada llamada a $ALV(x)$ tiene una probabilidad de éxito $p(x)$, el número esperado de pasadas por el ciclo es $1/p(x)$.
- Sea $T(x)$ el tiempo esperado antes de que $RepeatALV(x)$ tenga éxito.
- Para hacer un análisis correcto se debe considerar el tiempo esperado tomado por $ALV(x)$ en caso de éxito y en caso de fracaso.
- Sea $S(x)$ el tiempo esperado en caso de éxito y $F(x)$ el tiempo esperado en caso de fracaso.

- Con probabilidad $p(x)$, la primera llamada a $ALV(x)$ tiene éxito después de un tiempo esperado $S(x)$.
- Con probabilidad $1-p(x)$, la primera llamada a $ALV(x)$ fracasa después de un tiempo esperado $F(x)$.
- Después se retorna al punto de partida, y seguimos estando a un tiempo esperado $T(x)$ del éxito. Para este caso, el tiempo esperado total es $F(x) + T(x)$.

- Entonces $T(x)$ (la ecuación de recurrencia) está dado por:

$$T(x) = p(x)S(x) + (1 - p(x))(F(x) + T(x))$$

$$T(x) = S(x) + \frac{1 - p(x)}{p(x)}F(x)$$

- Por lo tanto, si la probabilidad de éxito es grande, el tiempo esperado para encontrar una respuesta correcta es bajo.
- Por otro lado, si la probabilidad de éxito es muy baja ($p \rightarrow 0$), entonces el tiempo esperado para encontrar una respuesta correcta puede ser muy prolongado.

- Búsqueda de un buen alumno.
- Quiero un alumno con nota ≥ 5.0
- $E(T(n)) \approx \frac{1}{p(\geq 5.0)}$

Algorithm 39 RepeatALV(x)

```
1: repeat  
2:   Elegir un alumno al azar;  
3: until nota  $\geq 5.0$   
4: return alumno; (El resultado que retorne siempre será correcto)
```

- Solución tipo Las Vegas. Si la probabilidad de encontrar un elemento mayoritario es muy baja, puede ser entonces que este algoritmo nunca responda.

Algorithm 40 LVMayoritario(int[] A, int n)

```
1: Input: A, matriz de enteros; n tamaño de A.
2: Output: verdadero si A es mayoritario.
3: while (1) do
4:   if (MayoritarioMC(A, n)) then
5:     return true;
6:   end if
7: end while
```
